Prompt-Aware WiFi Scheduling for Generative AI Services

Shugang Hao
Singapore University of Technology and Design (SUTD)
Singapore, Singapore
shugang_hao@sutd.edu.sg

Lingjie Duan*
Hong Kong University of Science and Technology
Guangzhou, China
lingjieduan@hkust-gz.edu.cn

Abstract

Mobile devices are increasingly accessing generative AI (GenAI) services through edge-deployed Large Language Models (LLMs) over WiFi networks, particularly in smart homes and smart offices. Devices transmit uplink prompts via OFDMA, a key WiFi 6 technology that enables concurrent access and improves network efficiency for dense GenAI traffic. The edge server batches these prompts to maximize inference efficiency. However, existing WiFi scheduling algorithms (e.g., max-weight and round-robin) largely ignore non-negligible LLM inference delay, random and diverse among different prompt inputs. Further, the batch inference delay is dictated by the longest prompt, coupling the service latency of co-batched stations and destabilizing device-side queues. In this work, we propose a novel prompt-aware WiFi scheduling for GenAI services. We formulate the scheduling problem as a Markov Decision Process (MDP) and characterize its fundamental computational barrier. We design a Renewal Drift-Plus-Penalty (RDPP) policy to balance backlogs against batching costs, using the drift to ensure queue stability and the penalty to constrain frame durations. We prove that RDPP is stable throughout an explicit offered-slack region and achieves near-optimal frame efficiency, the batching component of the average backlog, under an explicit tradeoff. To further address the impact of output response length on LLM inference, we extend to develop a robust R-RDPP policy that accounts for output-length uncertainty, with explicitly bounded quantization and uncertainty costs. We conduct experiments on an Nvidia A100 GPU (80GB) to measure LLM inference latency and derive closed-form functions of batch size and token lengths. NS-3 experiments further demonstrate great improvements of our proposed policies over existing WiFi schedulers in service delay, queue length, and tail latency.

CCS Concepts

• **Networks** → **Wireless local area networks**; • **Mathematics of computing** → **Mathematical optimization**.

Keywords

WiFi for Generative AI, OFDMA scheduling, LLM inference

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

ACM Reference Format:
Shugang Hao and Lingjie Duan. 2018. Prompt-Aware WiFi Scheduling for Generative AI Services. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX).* ACM, New York, NY, USA, 27 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Mobile devices are increasingly accessing generative AI (GenAI) services through edge-deployed Large Language Models (LLMs) over WiFi networks, where indoor environments (e.g., smart homes and smart offices) serve as the primary deployment domains (e.g., [31], [15]). In a typical edge GenAI workflow, a prompt is first transmitted from the station to a WiFi access point (AP) over the air interface via orthogonal frequency division multiple access (OFDMA), a key technology in WiFi 6 and 7 that significantly improves network efficiency, especially in dense environments for GenAI (e.g., [18], [21]). The AP then forwards the prompt over a wired backhaul to a nearby co-located edge LLM server, which generates the response to the request (e.g., [29], [17]).

Unlike traditional wireless traffic (e.g., web browsing or video streaming) where the delay bottleneck lies in the air-interface transmission (e.g., [26], [27]), this new GenAI traffic incurs non-negligible server-side inference delay compared to wireless transmission delay (e.g., [25], [31]), random and diverse among different prompt inputs. Further, the batch inference delay is dictated by the longest prompt (e.g., [15], [11]), coupling the service latency of co-batched stations and destabilizing device-side queues. Existing WiFi scheduling algorithms (e.g., max-weight style and round-robin) largely ignore these inference characteristics and make decisions based solely on queue backlogs and channel conditions (e.g., [28], [12], [24], [20]). One research question naturally arises:

- *Q1. How do current WiFi scheduling algorithms perform when facing generative AI services?*

Later we prove that existing methods can lead to unstable station queues, as a single long prompt bottlenecks the batch-inference process. This necessitates a new scheduling policy accounting for both air-interface transmissions and server-side inference costs. Few studies in the recent wireless literature analyze the end-to-end latency of GenAI services. Kim *et al.* [15] optimize decisions between local processing and server offloading by balancing cost, latency, and response quality for LLM services over 5G networks. Yang *et al.* [31] characterize end-to-end latency distributions using queueing models that capture both air-interface and LLM inference delays. Jin *et al.* [14] propose an end-to-end coordination framework CORA to improve the fraction of latency-critical inference requests meeting their latency targets. However, they primarily employ content-agnostic or bit-rate-centric models and fail to account for the non-linear coupling inherent in LLM batching, where a batch's service time is dictated by the maximum prompt length. Thus,

the interplay between OFDMA sub-carrier allocation and batch-inference efficiency remains unexplored.

Existing LLM works study inference delay and batching behavior from the server-side perspective. Yang *et al.* [32] apply queueing-theoretic abstractions to analyze how output length distribution affects inference delay. Guldogan *et al.* [11] propose binning-based batching to group requests by output length and improve throughput. Yu *et al.* [33] introduce Orca to improve GPU utilization under dynamic workloads. More recently, prompt-aware server-side schedulers have emerged that predict output lengths to approximate shortest-job-first ordering (e.g., [9]), further improving inference latency. However, all these works optimize inference latency strictly from a server-side perspective, ignoring source-side queue dynamics and providing no queue stability guarantees at the station side. Moreover, they do not account for wireless transmission delays, treating request arrivals as exogenous rather than as outcomes of wireless scheduling decisions. In practice, the AP's scheduling and the server's batch inference are tightly coupled, yet the impact of this interaction on station-side queue stability remains unexplored.

It is challenging to design efficient WiFi scheduling for GenAI services. First, prompt arrivals from mobile devices are stochastic in both input length and arrival time, making it difficult to balance the immediate relief of queue backlogs against the heavy inference delays. Second, the edge LLM batching mechanism induces a strong coupling effect, where the inference delay is dictated by the maximum input and output lengths within a batch. Thus, the scheduling of one station's prompt directly affects the service of others. This max-based cost fundamentally differs from classical air-interface scheduling, where server delay is typically negligible and user performance remains independent, making it difficult to adapt traditional models to handle this non-linear coupling. The second research question is thus:

- *Q2. How to design an efficient prompt-aware WiFi scheduling policy for GenAI services to stabilize queue backlogs?*

We summarize our key novelty and main results as follows.

- *Prompt-aware WiFi scheduling for generative AI services:* In this paper, we study a novel prompt-aware WiFi scheduling for generative AI services. Unlike existing WiFi scheduling approaches that consider only air-interface transmission and ignore server-side inference delay, we explicitly incorporate LLM inference characteristics induced by prompt batching. We aim to provide new analytical insights into *how to design efficient scheduling schemes for generative AI services.*
- *MDP formulation and computational barrier:* We prove that existing WiFi scheduling algorithms (e.g., max-weight and round-robin) can lead to unstable queues at the station side under GenAI workloads. We formulate prompt-aware OFDMA scheduling as a Markov Decision Process (MDP). We show that computing an optimal MDP policy faces a computational barrier: decision epochs embed a batch-partitioning problem with no known polynomial-time algorithm.
- *RDPP policy with theoretical guarantees:* We propose a Renewal Drift-Plus-Penalty (RDPP) scheduling policy that jointly optimizes queue backlogs and batching-induced LLM inference costs, where the drift acts to stabilize the queues by clearing accumulated backlogs and the penalty restrains excessive frame durations. We prove that RDPP is stable throughout an explicit offered-slack region and achieves near-optimal frame efficiency, the batching component of the long-run average backlog, with gap $O(1/V)$ for a tunable $V > 0$ trading efficiency against an $O(V)$ backlog.
- *Extension to output-length uncertainty:* We further extend RDPP to account for uncertainty in output response length, which affects decoding latency and is unknown at scheduling time. We propose a robust R-RDPP policy and prove stability and efficiency guarantees with explicitly bounded quantization and uncertainty costs. We conduct experiments on an Nvidia A100 GPU (80GB) to measure LLM inference latency, and derive closed-form functions of batch size and token lengths. NS-3 experiments further demonstrate great improvements of our proposed policies over existing methods in queue length, service delay, and tail latency.

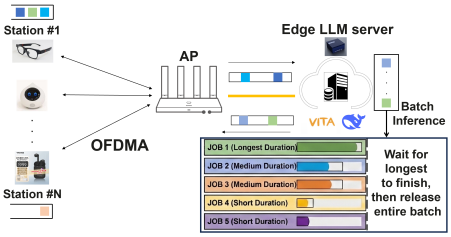

Figure 1: System model of WiFi for generative AI services.

The rest of this paper is organized as follows. Section 2 introduces the system model, problem formulation, and benchmark policies' instability under GenAI workloads. Section 3 presents the MDP formulation and characterizes its computational barrier. Section 4 details our RDPP policy design and provides theoretical guarantees on stability and near-optimality. Section 5 extends RDPP to handle output length uncertainty. Section 6 conducts experiments to validate our theoretical findings. Section 7 finally concludes the paper. Due to the page limit, we skip some detailed proofs but still introduce sketches in the main text. Please refer to online technical report [13] for details.

2 System Model and Problem Formulation

In this section, we first introduce our system model of WiFi for GenAI services. We then give our problem formulation and the instability of existing WiFi scheduling algorithms.

2.1 System Model of WiFi for GenAI

As shown in Figure 1, we consider a time-slotted WiFi-based edge AI system, where an access point (AP) schedules and serves uplink prompt transmissions from N stations (e.g., smart-home devices like smartphones and smart security cameras, or office devices such as laptops and smart displays) over the air interface. Meanwhile, it forwards received prompts' packets via a wired backhaul to a nearby co-located edge LLM server for inference and response generation. In each time slot t, each station $i \in [N] := \{1, \ldots, N\}$ generates a new prompt with input length $\ell_i^{\text{in}} \in \{L_0^{\text{in}}, L_1^{\text{in}}, \ldots, L_I^{\text{in}}\}$ (in terms of packet number), where $L_0^{\text{in}} = 0$ means no prompt generated and $0 < L_1^{\text{in}} < \cdots < L_I^{\text{in}}$ with $Pr(\ell_i^{\text{in}} = L_j^{\text{in}}) = \alpha_j \geq 0$ and

$\sum_{j=0}^{I} \alpha_j = 1$ i.i.d. across stations and time slots. This i.i.d. assumption reflects uncoordinated user behavior, where diverse devices generate heterogeneous tasks stochastically based on immediate user needs. Thus, the total expected packet arrival rate across the system is given by $\Lambda = \sum_{i=1}^{N} \mathbb{E}[\ell_i^{\text{in}}] = N \sum_{j=1}^{I} \alpha_j L_j^{\text{in}}$. In Section 5, we extend to consider a continuous interval of prompt input lengths.

Each station maintains a buffer to store prompts awaiting transmission and inference, allowing flexible service ordering as determined by the scheduling policy. Buffering occurs only at the station side. The AP and the edge LLM server do not maintain persistent buffers and process prompts immediately upon reception. By making uplink scheduling aware of the server's real-time workload, the system effectively maintains a virtually centralized queue at the station side, avoiding multi-hop queueing delays. In practice, the AP obtains station-side buffer information through the Buffer Status Report (BSR) mechanism (e.g., WiFi 6 and 7), where each station piggybacks its queue occupancy in uplink trigger-based frames, enabling the AP to make prompt-aware scheduling decisions with up-to-date backlog and prompt-length information.

The AP employs OFDMA to schedule uplink transmissions. In each time slot, the AP can allocate up to $k \geq 1$ resource units (RUs) to serve up to k stations in parallel. When station i is scheduled, one prompt from its buffer is transmitted to the AP and forwarded to the edge LLM server. A length-ℓ^{in} prompt requires ℓ^{in} slots to complete transmission. We consider time-homogeneous channel conditions across stations to focus on the fundamental interaction between prompt-aware scheduling and batch inference delay. This is consistent with the standard approach in the wireless scheduling theory literature for GenAI services (e.g., [30], [4], [35]).

The edge LLM server processes incoming prompts using static batching, widely adopted in the LLM inference literature (e.g., [33], [32]). In each service round, the server groups multiple prompts into a batch and performs joint inference, where the batch size is constrained by hardware resources. We consider $B = k$ to reflect a common deployment scenario where the edge server's batching capacity is provisioned to match the OFDMA parallelism of the WiFi AP. This alignment ensures that all prompts delivered in a single wireless frame can be processed in a single inference round, avoiding unnecessary queueing at the server-side buffer. Under static batching, the server provisions a fixed batch shape of size $B = k$ and pads unfilled slots, so a round's latency is governed by the provisioned size and its longest prompt.

A key property of LLM batch inference is that the inference delay depends on the longest prompt input and the longest response output in the batch (e.g., [32], [33]). Let $\ell_{\max}^{\text{in}} := \max_{i \in \mathcal{B}} \ell_i^{\text{in}}$ and $\ell_{\max}^{\text{out}} := \max_{i \in \mathcal{B}} \ell_i^{\text{out}}$, where ℓ_i^{out} denotes the output response length generated by the LLM for station i. We model the Time-To-First-Token (TTFT) as follows:

$$\text{TTFT} = f(B, \ell_{\max}^{\text{in}}), \tag{1}$$

where $f(\cdot)$ is known and nondecreasing in both arguments, capturing the empirical observation that prefill latency scales with the maximum input length in the batch (e.g., [25], [7]). The decode-phase latency, characterized by the inter-token latency or Time-Per-Output-Token (TPOT), depends on $\ell_{\max}^{\text{out}}$ and is modeled by a known and nondecreasing function $g(\cdot)$ as follows:

$$\text{TPOT} = g(B, \ell_{\max}^{\text{in}}, \ell_{\max}^{\text{out}}). \tag{2}$$

We define the frame duration of a batch as

$$\tau := \ell_{\max}^{\text{in}} + f(B, \ell_{\max}^{\text{in}}) + g(B, \ell_{\max}^{\text{in}}, \ell_{\max}^{\text{out}}) \times (\ell_{\max}^{\text{out}} - 1), \tag{3}$$

capturing the total time from the start of OFDMA transmission to the completion of response generation. Note that the latency functions $f(\cdot)$ and $g(\cdot)$ are intrinsic to the LLM inference hardware and remain invariant to wireless channel conditions, as they depend solely on the batch size and token lengths processed by the GPU. In practice, both functions can be estimated beforehand via offline profiling on the edge LLM server. In Section 6.1, we conduct experiments on an Nvidia A100 GPU to measure TTFT and TPOT, and derive closed-form functions of batch size and token lengths.

Table 1: Summary of Key Notations

Symbol	Description
N, k, B	Num. of STAs, OFDMA RUs, batch size ($B{=}k$)
$t, \Delta t$	Time slot index; slot duration
$\ell_i^{\text{in}}(t), \ell_i^{\text{out}}$	Input prompt / output response length of STA i
$L_0^{\text{in}}, \ldots, L_I^{\text{in}}$	Discrete prompt length levels ($L_0^{\text{in}}{=}0$: no arrival)
α_j, Λ	Prompt length probability; total arrival rate
$Q_i(t), Q_i^+(t)$	Backlog / post-arrival backlog at STA i
$\boldsymbol{Q}(t)$	Vector of all station backlogs
$\mathcal{B}(t), n_{i,j}(t)$	Buffer composition; num. prompts of length L_j^{in}
$r(t)$	Residual service time of ongoing batch
$\ell_{\max}^{\text{in}}, \ell_{\max}^{\text{out}}$	Max input / output length in a batch
$f(\cdot), g(\cdot)$	TTFT (prefill latency); TPOT (decode latency)
$\tau(L)$	Frame duration
$\mathcal{S}_t^{\text{sched}}, U_i(t)$	Scheduled station set; scheduling indicator
$H_i^{\text{sel}}(t), m(t)$	Selected prompt length; num. scheduled STAs
$\ell_{\max}^{\text{sel}}(t)$	Max selected prompt length in batch
S_t	MDP state: $(\boldsymbol{Q}(t), \mathcal{B}(t), r(t))$
$c^\star, \bar{Q}^\pi$	Optimal average cost; avg. backlog under π
$\mu^\star, C$	Max service rate; capacity region
$L, \mathcal{L}$	Batch threshold; feasible threshold set
$\ell_i(L), S(L)$	Longest prompt $\leq L$ at STA i; selected batch
$V, \Phi_V(L)$	Tunable parameter; RDPP score
$\delta, \mathcal{G}_\delta$	Quantization resolution; grid
$Q_\delta(\cdot), I_\delta$	Quantization map; num. grid points
$\hat{\tau}(L), \hat{\sigma}_\tau(L)$	Estimated frame duration; its std. dev.
η, Δ_g	Robustness param.; decode duration spread
$\Phi_V^\eta(L), \mu_\delta^\star$	R-RDPP score; grid-sampled service bound

2.2 Problem Formulation and Criteria

Following the WiFi scheduling literature (e.g., [28], [10], [6]), we aim to minimize queueing backlog at the station or device side by appropriately designing wireless scheduling decisions. Let $Q_i(t)$ denote the packet number in station i's buffer at the beginning of slot t. A scheduling policy π determines, at each slot, which subset of stations is scheduled for OFDMA transmission. Our objective is to minimize the long-run average total buffer length:

$$\min_{\pi} \quad \bar{Q}^\pi := \limsup_{T \to \infty} \frac{1}{T} \mathbb{E}_\pi \left[\sum_{t=0}^{T-1} \sum_{i=1}^{N} Q_i(t) \right]. \tag{4}$$

By Little's law, minimizing $\bar{Q}^\pi$ is equivalent to minimizing the average queueing delay, making this formulation appropriate for interactive generative AI services.

Before formalizing the stability analysis, we first define the notion of capacity region and then characterize the stability.

Definition 2.1 (Capacity Region). The capacity region C is defined as the set of total packet arrival rates Λ for which there exists a scheduling policy that stabilizes the system. The *offered-slack region* $C_{\text{off}} \subseteq C$ consists of the rates that admit a uniform service margin: $\Lambda \in C_{\text{off}}$ if some stationary randomized policy, choosing batches independently of the queue state and padding unfilled slots with dummy prompts, offers every station service faster than its packets arrive by a fixed margin $\varepsilon > 0$, at every nonempty buffer composition.

Under the homogeneous arrivals of Section 2.1, each station's packet arrival rate is Λ/N, so Λ parameterizes the per-station arrival-rate vector; the offered-slack condition above is stated per station.

Definition 2.2 (Stability). The system is *stable* under policy π if it leads to a finite expected queue length $\bar{Q}^{\pi} < \infty$. A policy that achieves stability for all arrival rates strictly inside the capacity region is called *throughput-optimal* (e.g., [28], [5]).

Later we construct stabilizable arrival rates in C at which each benchmark is unstable, while our policies are stable for every rate in C_{off}.

2.3 Preliminary: Benchmark Schemes

In this subsection, we introduce the commonly used max-weight style and round-robin scheduling as benchmark schemes to examine their performance under generative AI workloads.

At each scheduling opportunity, max-weight style scheduling (e.g., [28]) assigns a weight to each station proportional to its queue backlog and selects the set of stations that maximizes the total weight subject to resource constraints. In an OFDMA system with k parallel resource units, this reduces to selecting the k stations with the largest queue lengths. More recent variants incorporate multi-agent reinforcement learning (MFMAPPO [12]) and deep hierarchical reinforcement learning (DHRL [24]) to learn scheduling weights for 802.11ax OFDMA, yet they still rely on queue backlogs and channel conditions without accounting for batch inference coupling. Round-robin scheduling is widely used for its simplicity and fairness (e.g., WiFi 6, 7 [1], [19]). The policy cycles through stations in a fixed order and allocates transmission opportunities evenly across users, independent of queue backlogs or prompt characteristics. Both benchmarks are length-oblivious and serve each selected station's head-of-line prompt in first-in-first-out order. Neither is guaranteed to stabilize every stabilizable GenAI workload.

Lemma 2.3. *Max-weight style and round-robin scheduling can fail on stabilizable systems: for each policy, there exist system instances that an explicit stationary policy stabilizes, yet on which the policy in question is unstable. Here*

$$\mu^{\star} := \max_{\ell \in \{L_1^{\text{in}}, \ldots, L_I^{\text{in}}\}} \frac{k\ell}{\ell + f(k,\ell)} \tag{5}$$

denotes the maximum achievable service rate over all feasible batch compositions, and $k\ell/(\ell + f(k,\ell))$ represents the maximum aggregate payload normalized by the joint transmission-inference duration.

Since a frame whose longest served prompt has length ℓ delivers at most $k\ell$ packets over at least $\ell + f(k,\ell)$ slots, every stabilizable rate satisfies $\Lambda \le \mu^{\star}$; $\mu^{\star}$ is thus an outer bound on the capacity region C. The root cause of instability under both max-weight and round-robin policies is the inefficient mixing of long and short prompts, which triggers a max-based inference delay. In the constructed max-weight instance, $N = k$, so every nonempty station is selected; instability arises from its length-oblivious FIFO service, which transmits a long prompt as soon as it reaches a head-of-line position. Round-robin similarly lacks prompt-length coordination. Since inference time is governed by the longest request, in the constructed instances this persistent mixing inflates service times and reduces the effective service rate, causing queues to grow unboundedly. The counter-instances use superlinear (max-weight) and linear (round-robin) TTFTs; the same mixing inefficiency drives the benchmarks' degradation in Section 6.2.

Since existing WiFi schedulers are highly inefficient under GenAI workloads, this motivates prompt-aware OFDMA scheduling that jointly accounts for wireless transmission and LLM inference to ensure queue stability. In Sections 3–4, we focus on the prefill latency by setting $g \equiv 0$ (e.g., real-time interactive chatbots and streaming voice assistants). We incorporate the analysis of TPOT with output-length uncertainty (e.g., code generation) in Section 5.

3 MDP Formulation and Computational Barrier

In this section, we formulate the prompt-aware OFDMA scheduling problem for WiFi-based GenAI services as a Markov Decision Process (MDP), which captures stochastic prompt arrivals, batch-dependent LLM inference delay, and sequential scheduling decisions. We then characterize the optimality conditions of this MDP and show that computing an optimal policy faces a fundamental computational barrier.

3.1 MDP Formulation

The system evolves in discrete time slots where prompt arrivals occur at the beginning. In the following, we illustrate our design of state, action, transition and cost in detail.

State. In time slot t, the system state is defined as

$$S_t := \big(\boldsymbol{Q}(t), \boldsymbol{\mathcal{B}}(t), r(t)\big).$$

We denote $\boldsymbol{Q}(t) = (Q_1(t), \ldots, Q_N(t))$ as buffer backlogs at all stations and $Q_i(t) \in \mathbb{Z}_{\ge 0}$ represents the total number of buffered prompts' packets at station i. We represent the buffer composition state at time t by $\boldsymbol{\mathcal{B}}(t) := \{n_{i,j}(t) : i \in [N], j \in [I]\}$, where $n_{i,j}(t) \in \mathbb{Z}_{\ge 0}$ denotes the number of buffered prompts at station i with input length L_j^{in}. $r(t) \in \mathbb{Z}_{\ge 0}$ is the residual service time of the ongoing batch, capturing the remaining service time before the edge server releases the responses. The server is idle only if $r(t) = 0$ and a new scheduling decision can be made. Recall that $\ell_i^{\text{in}}(t)$ denotes the input length (in packets) of the prompt arriving at station i at the beginning of slot t. The post-arrival backlog is

$$Q_i^{+}(t) = Q_i(t) + \ell_i^{\text{in}}(t).$$

Action. At a decision epoch (i.e., when $r(t) = 0$ and at least one station has $Q_i^{+}(t) > 0$), the scheduler selects a subset

$$S_t^{\text{sched}} \subseteq \{1, \ldots, N\}, \qquad |S_t^{\text{sched}}| \le k,$$

and chooses one prompt's packets to transmit from each selected station. If $r(t) > 0$ or all stations are empty after arrivals, the only admissible action is to remain idle. Formally, a declared action is $a_t = (L_t, S_t^{\text{sched}})$ with declared threshold L_t, where S_t^{sched} may be empty: an empty selection initiates a dummy-only provisioned frame at L_t, incurring the declared frame duration with zero queue service. In Sections 3–4, every declared threshold satisfies $L_t \in \mathcal{L} = \{L_1^{\text{in}}, \ldots, L_I^{\text{in}}\}$; a threshold-free batch is equivalently represented by declaring its realized maximum prompt length, which belongs to $\mathcal{L}$.

Transition. Define the scheduling indicator $U_i(t) = \mathbf{1}\{i \in S_t^{\text{sched}}\}$ with $\sum_{i=1}^{N} U_i(t) \le k$. Denote $H_i^{\text{sel}}(t)$ as the input length of the selected prompt. The buffer dynamics evolves as

$$Q_i(t+1) = Q_i^+(t) - U_i(t)\, H_i^{\text{sel}}(t).$$

If a new batch is initiated at slot t, define

$$m(t) := \sum_{i=1}^{N} U_i(t), \qquad \ell_{\max}^{\text{sel}}(t) := \max_{i:U_i(t)=1} H_i^{\text{sel}}(t).$$

The corresponding frame duration is

$$\tau(t) = \ell_{\max}^{\text{sel}}(t) + f\big(k, \ell_{\max}^{\text{sel}}(t)\big),$$

where the TTFT is evaluated at the provisioned batch size k, since the server reserves a padded batch shape of size $B = k$ (Section 2.1). Each action also carries a declared maximum length $L \ge \ell_{\max}^{\text{sel}}(t)$ and the frame is reserved for $\tau(t) = L + f(k, L)$, with threshold-free policies implicitly declaring $L = \ell_{\max}^{\text{sel}}(t)$ (benchmarks unaffected); in Section 5 the random decode term runs the frame to its realized duration. The residual service time updates as

$$r(t+1) = \begin{cases} \tau(t) - 1, & \text{if } r(t) = 0 \text{ and } Z(t) = 1, \\ \max\{r(t) - 1, 0\}, & \text{otherwise,} \end{cases}$$

where the frame-start indicator $Z(t)$ equals 1 when the action declares a frame: either a real batch ($m(t) \ge 1$) or a dummy-only declared action ($m(t) = 0$), which reserves $\tau(t)$ at the declared threshold with zero queue service; $Z(t) = 0$ for an ordinary idle slot.

Objective. The per-slot cost is defined as the total backlog,

$$c(S_t, a_t) = \sum_{i=1}^{N} Q_i(t).$$

For a policy π, the long-run average backlog is

$$c^{\pi} = \limsup_{T\to\infty} \frac{1}{T} \mathbb{E}_{\pi}\left[\sum_{t=0}^{T-1} c(S_t, a_t)\right]. \tag{6}$$

Note that the c^{π} in (6) is consistent with the objective $\bar{Q}^{\pi}$ defined in (4). The analysis uses the post-arrival convention. Formally, the post-arrival state is $S_t^+ := (\boldsymbol{Q}^+(t), \boldsymbol{\mathcal{B}}^+(t), r(t))$, where $\boldsymbol{Q}^+(t)$ and $\boldsymbol{\mathcal{B}}^+(t) := \{n_{i,j}^+(t)\}$ count backlogs and buffered prompts after the slot-t arrivals, and the stage cost is $c^+(S_t^+, a_t) := \sum_i Q_i^+(t)$. Since $\mathbb{E}[\sum_i Q_i^+(t)] = \mathbb{E}[\sum_i Q_i(t)] + \Lambda$ for every policy and every slot, the resulting objective equals (6) plus the policy-independent constant Λ, with identical optimizers. The ACOE of Section 3.2 and Appendices C–F are stated on the chain $\{S_t^+\}$ with cost c^+. Our objective is to find a policy π minimizing c^{π}. The backlog objective in (6) is jointly shaped by both wireless transmission and LLM batch inference. Since new scheduling decisions are made only when the server is idle, the batch inference delay directly governs how frequently station queues can be drained, causing arriving prompts to accumulate during server-busy periods.

3.2 Computational Barrier

We first characterize the optimality conditions of the above MDP. Under standard regularity assumptions for countable-state average-cost MDPs (e.g., [2]), there exists an optimal average cost $c_+^{\star}$ and a differential cost function $h : \mathcal{S} \to \mathbb{R}$ satisfying the average-cost optimality equation (ACOE), stated on the post-arrival pair (S^+, c^+) of Section 3.1:

$$h(S^+) + c_+^{\star} = \min_{a\in\mathcal{A}(S^+)} \left\{c^+(S^+, a) + \mathbb{E}\big[h(S^{+\prime}) \mid S^+, a\big]\right\}, \tag{7}$$

where $\mathcal{A}(S^+)$ denotes the admissible action set in state S^+, and $S^{+\prime}$ is the next post-arrival state under the transition dynamics. Although the ACOE characterizes optimal policies, solving it exactly must contend with an exponentially large state space and an embedded batch-partitioning problem for which no polynomial-time algorithm is known, as shown below.

Lemma 3.1 (Computational Barrier). *Solving the ACOE must contend with a state space whose cardinality grows exponentially in N and with decision epochs that embed a bounded parallel-batch clearing problem: partitioning buffered prompts into batches of size at most k under max-based frame durations to minimize $\sum_i p_i S_i$, for which no polynomial-time algorithm is known and whose exact complexity is, to the best of our knowledge, unresolved [3, 8, 34].*

Appendix B details the embedding and complexity status; we claim NP-hardness neither for the clearing problem nor for the average-cost MDP. These computational obstacles motivate a directly analyzable low-complexity scheduling policy with provable performance guarantees, presented in the next section.

4 RDPP: Stability and Near-Optimal Frame Efficiency

In this section, we develop a low-complexity scheduling policy with provable performance guarantees. We first introduce the design principles of the Renewal Drift-Plus-Penalty (RDPP) policy by exploiting the renewal structure induced by batch processing. We then establish its theoretical guarantees on stability under offered-service slack and near-optimal frame efficiency.

4.1 RDPP Policy Design

As shown in Section 3.2, computing an optimal scheduling policy has no known polynomial-time algorithm and faces an exponentially large state space. We therefore seek a low-complexity policy that stabilizes the system under an explicit slack condition and achieves near-optimal frame efficiency. Our design leverages the renewal structure induced by batch processing. Scheduling decisions are made only at renewal epochs (i.e., when $r(t) = 0$), while the system evolves deterministically during each busy frame.

At a renewal epoch t, the scheduler observes the post-arrival state $\{Q_i^+(t), n_{i,j}^+(t)\}$. To reduce combinatorial complexity, we parameterize each candidate batch by its maximum input-length

Algorithm 1 RDPP Scheduling Policy

Require: Stations N, resource units k, prompt-length thresholds $\mathcal{L} = \{L_1^{\text{in}}, \ldots, L_I^{\text{in}}\}$, tradeoff parameter $V > 0$
Ensure: Scheduling decisions $\{L^\star, S(L^\star)\}$ at each renewal epoch
for each renewal epoch (server idle $r(t) = 0$, some buffer nonempty $Q_i^+(t) > 0$) **do**
  for each candidate max-length threshold $L \in \mathcal{L}$ **do**
    Frame duration: $\tau(L) \leftarrow L + f(k, L)$
    Longest eligible prompt at each STA: $\ell_i(L) \leftarrow \max\{\ell \le L :$ prompt of length ℓ at STA $i\}$
    Per-station weight (backlog relief × prompt size): $w_i(L) \leftarrow (Q_i^+(t) + V(\tau(L) - 1)) \cdot \ell_i(L), \forall i$
    Select batch with highest weights: $S(L) \leftarrow$ up to k STAs with largest $w_i(L)$ among those with $\ell_i(L) > 0$
    Evaluate per-slot reward: compute $\Phi_V(L)$ via (8)
  Pick best threshold: $L^\star \leftarrow \arg\max_{L \in \mathcal{L}} \Phi_V(L)$
  Transmit batch $S(L^\star)$ via OFDMA; update queues (if $S(L^\star) = \emptyset$, run a dummy-only provisioned frame of type $L^\star$, no queue service)
  Begin batch inference: $r(t+1) \leftarrow \tau(L^\star) - 1$; all responses released together at $t + \tau(L^\star)$

threshold L. Fixing L determines the frame duration τ in (3) as

$$\tau(L) := L + f(k, L),$$

and restricts eligible prompts to those with length not exceeding L. For each station i, define

$$\ell_i(L) := \max\{\ell \le L : \text{a prompt of length } \ell \text{ exists at station } i\},$$

with $\ell_i(L) = 0$ if no such prompt exists. For each feasible threshold L, we construct a batch $S(L)$ by selecting up to k stations with the largest weights, denoted as follows:

$$\big(Q_i^+(t) + V(\tau(L) - 1)\big)\ell_i(L),$$

where $V>0$ is a tunable parameter. The RDPP score is then

$$\Phi_V(L) := \frac{\sum_{i \in S(L)} \big(Q_i^+(t) + V(\tau(L) - 1)\big)\ell_i(L)}{\tau(L)} - \frac{V\Lambda}{2}(\tau(L) - 1). \quad (8)$$

The structure of $\Phi_V(L)$ reflects a renewal drift-plus-penalty principle. The numerator rewards backlog reduction weighted by prompt length, while division by $\tau(L)$ converts the reward into a per-slot rate. The final term penalizes long frames that allow excessive arrival accumulation. The parameter V controls the tradeoff between backlog responsiveness and frame efficiency. A threshold with no eligible prompt ($S(L) = \emptyset$) is admissible: it denotes a dummy-only provisioned frame of type L, occupying $\tau(L)$ slots with zero queue service.

Given the renewal structure of the system and the threshold-based parameterization of feasible batches, we now formally present the complete RDPP scheduling procedure in Algorithm 1.

The per-renewal computational complexity of RDPP is $O(I\,N \log k)$, since at most I candidate thresholds are evaluated and, for each threshold, selecting up to k stations with largest weights can be implemented using a size-k heap over N stations.

4.2 Theoretical Guarantees

Having specified the RDPP policy, we now analyze its stability properties. We first show RDPP stabilizes the system for every arrival rate in the offered-slack region C_{off} of Definition 2.1.

Lemma 4.1 (Stability under Offered-Service Slack). *For any $V > 0$, RDPP stabilizes the system for all arrival rates in the offered-slack region C_{off} of Definition 2.1.*

Lemma 4.1 shows that RDPP stabilizes the entire offered-slack region C_{off}, the sufficient stability region our analysis certifies. Its proof rests on a threshold-dominance property: any feasible batch is score-dominated by the threshold batch at its own maximum length (Lemma C.4, Appendix C). Next, we quantify RDPP's batching efficiency against the best stationary benchmark and give an exact representation of its total-cost gap.

Call a state-blind stationary randomized rule offering each selected station its longest eligible prompt (dummies padding absent ones) an *offered-service comparator*; slack ε means it offers every station rate $\lambda_i^{\text{pkt}} + \varepsilon$ at every nonempty composition.

Theorem 4.2 (Near-Optimal Frame Efficiency). *Define the frame-inefficiency rate of an action with declared threshold L and served packet total d as $\psi := \Lambda(\tau(L) - 1)/2 - (\tau(L) - 1)\,d/\tau(L)$, and let $\psi^\star$ be the least worst-composition average inefficiency over offered-service comparators with positive slack. For every $V > 0$, RDPP is stable, its long-run average frame-inefficiency satisfies $\bar{\psi}^{\text{RDPP}(V)} \le \psi^\star + O(1/V)$, and the time-average total backlog is finite, with a bound of order $O(V)$.*

Theorem 4.2 certifies RDPP at the quantity its score controls: every policy's expected frame cost equals its frame duration times the sum of its standing backlog and ψ (Lemma C.3), and V trades batching efficiency against backlog in the classical $[O(1/V), O(V)]$ form; a moderate V balances the two, as in Section 6. Appendix D also gives an exact ACOE representation of the total-cost gap, which does not vanish with V.

5 Extension to Output Response Uncertainty

In this section, we first extend the system model to account for output-response uncertainty related to TPOT in (2), and extend to continuously distributed input prompt lengths beyond the discrete set in Section 2.1. We then design a robust quantized scheduling policy and establish its theoretical performance guarantees.

5.1 Extension of System Model to Output Response Uncertainty

With output lengths taken into account, the frame duration τ of a batch $\mathcal{B}$ in (3) becomes

$$\tau(B, \ell_{\max}^{\text{in}}, \ell_{\max}^{\text{out}}) = \ell_{\max}^{\text{in}} + f(B, \ell_{\max}^{\text{in}}) + D(B, \ell_{\max}^{\text{in}}, \ell_{\max}^{\text{out}}),$$

where $\ell_{\max}^{\text{in}} = \max_{i \in \mathcal{B}} \ell_i^{\text{in}}$, $\ell_{\max}^{\text{out}} = \max_{i \in \mathcal{B}} \ell_i^{\text{out}}$, and $D(B, \ell^{\text{in}}, \ell^{\text{out}}) := g(B, \ell^{\text{in}}, \ell^{\text{out}}) \times (\ell^{\text{out}} - 1)$ denotes the total decode duration, consistent with (3). The first two terms correspond to the transmission plus prefill duration and are known at the renewal epoch, whereas the decode term depends on output lengths that are revealed only after batch selection. This creates an information asymmetry that the scheduler must commit to a batch while part of its service time is still random.

We assume each prompt's output length ℓ^{out} is drawn from a bounded-support distribution that may depend on its input length, and output lengths are conditionally independent across prompts

given input lengths. The variability induced by output uncertainty is captured by

$$\Delta_g := \max_{b\le k,\ \ell^{\text{in}}} \left[\max_{\ell^{\text{out}}} D(b,\ell^{\text{in}},\ell^{\text{out}}) - \min_{\ell^{\text{out}}} D(b,\ell^{\text{in}},\ell^{\text{out}}) \right],$$

which measures the worst-case spread of the decode duration due to unknown output response lengths. Intuitively, a larger Δ_g implies that two batches with identical observed input profiles can still incur very different decode times, making purely input-based scheduling less reliable. We further generalize input prompt lengths from a finite set to a bounded continuum $\ell^{\text{in}} \in [\ell^{\text{in}}_{\min}, \ell^{\text{in}}_{\max}]$, drawn from a general distribution; from here on, $\ell^{\text{in}}_{\max}$ denotes this support endpoint rather than a batch maximum, which Appendices E–F write as $\tilde{\ell}$. Bounded support ensures uniformly bounded frame durations and preserves the renewal drift structure. However, continuous lengths make the state description more complex. More importantly, they induce an infinite action space because the batch threshold can take any real value. This motivates quantization to recover a finite decision space with controlled approximation error.

Assumption 1 (Regularity of the latency maps). *On the bounded input interval, $f(k,\cdot)$ and $D(k,\cdot,\ell^{\text{out}})$ are nondecreasing and Lipschitz, with constants L_f and L_D uniformly in ℓ^{out}; we write $\kappa := 1+L_f+L_D$, and $\tau_{\min} \ge 1$, $\bar{\tau} < \infty$ for the minimum and maximum frame durations.*

5.2 Robust RDPP and Performance Guarantees

To handle both continuous input lengths and output-induced service-time uncertainty, we design a robust quantized extension of RDPP. The key idea is to approximate the continuous prompt-length space using a finite grid while incorporating uncertainty-aware scoring to hedge against unknown decode durations.

We fix a resolution $\delta > 0$ and define the uniform grid

$$\mathcal{G}_\delta := \left\{ \ell^{\text{in}}_{\min} + m\delta : m = 0,1,\dots, \left\lfloor \frac{\ell^{\text{in}}_{\max} - \ell^{\text{in}}_{\min}}{\delta} \right\rfloor \right\}.$$

For any prompt with true input length $\ell^{\text{in}} \in [\ell^{\text{in}}_{\min}, \ell^{\text{in}}_{\max}]$, define the quantization map

$$Q_\delta(\ell^{\text{in}}) := \max\{L \in \mathcal{G}_\delta : L \le \ell^{\text{in}}\}.$$

Then $Q_\delta(\ell^{\text{in}}) \le \ell^{\text{in}} < Q_\delta(\ell^{\text{in}}) + \delta$, and each prompt is assigned to one of $I_\delta = |\mathcal{G}_\delta| = O(1/\delta)$ discrete types. Quantization is used only for scheduling decisions (e.g., threshold selection and batch construction), while the actual queue evolution and served packet counts are computed using the true input lengths.

Since decode durations are unknown at scheduling time, R-RDPP prices frames under the *provisioned reference profile*: $\hat{D}(L)$ denotes the expected decode duration of k i.i.d. outputs drawn conditional on input length L, and $\hat{\tau}(L) := L + f(k,L) + \hat{D}(L)$, with reference standard deviation $\hat{\sigma}_\tau(L) \le \Delta_g$. Both depend on L alone, so the weights and score of Algorithm 2 share one estimate with no circular dependence on $S(L)$.Its bias relative to the realized batch, at most $\Delta_g + \kappa\delta$, is propagated through all guarantees (Lemma E.1); profile estimates are treated as exact. To hedge against output uncertainty,

Algorithm 2 R-RDPP: Robust Extension with Output Uncertainty

Require: Stations N, resource units k, quantization resolution $\delta > 0$, robustness parameter $\eta \ge 0$, tradeoff parameter $V > 0$
Ensure: Scheduling decisions $\{L^\star, S(L^\star)\}$ at each renewal epoch

Discretize prompt lengths into grid $\mathcal{G}_\delta$; collect runtime estimates of the reference-profile decode mean $\hat{D}(\cdot)$ and its variability $\hat{\sigma}_\tau(\cdot)$
for each renewal epoch (server idle $r(t) = 0$, some buffer nonempty $Q_i^+(t) > 0$) **do**
  Map each prompt to its grid type (largest grid point not exceeding it): $\ell_i^{\text{in}} \mapsto Q_\delta(\ell_i^{\text{in}})$, $\forall i$
  for each candidate threshold $L \in \mathcal{G}_\delta$ **do**
    Estimated frame duration (provisioned at batch size k): $\hat{\tau}(L) \leftarrow L + f(k,L) + \hat{D}(L)$
    Longest eligible prompt at each STA: $\ell_i(L) \leftarrow \max\{\ell :$ prompt of length ℓ at STA i, $Q_\delta(\ell) \le L\}$
    Per-station weight: $w_i(L) \leftarrow (Q_i^+(t) + V(\hat{\tau}(L)-1)) \cdot \ell_i(L)$, $\forall i$
    Select batch with highest weights: $S(L) \leftarrow$ top-k STAs by $w_i(L)$ among those with $\ell_i(L) > 0$
    Evaluate uncertainty-aware reward: compute $\Phi_V^\eta(L)$ via (9)
  Pick best threshold: $L^\star \leftarrow \arg\max_{L\in\mathcal{G}_\delta} \Phi_V^\eta(L)$
  Transmit batch $S(L^\star)$ via OFDMA; update queues with *true* (unquantized) ℓ_i^{in}
  If $S(L^\star) = \emptyset$, run a dummy-only provisioned frame of type $L^\star$ with output drawn from the reference profile at $L^\star$; otherwise, if every selected true length is below $L^\star$, sequence-pad one selected prompt's input to $L^\star$ for computation only (saturated scoring, Appendix F)
  Begin batch inference: $r(t+1) \leftarrow \tau_{\text{actual}}(L^\star) - 1$; all responses released together

we inflate the effective duration and define the score

$$\Phi_V^\eta(L) = \frac{\sum_{i\in S(L)} \left(Q_i^+(t) + V(\hat{\tau}(L)-1)\right)\ell_i(L)}{\hat{\tau}(L) + \eta\hat{\sigma}_\tau(L)} - \frac{V\Lambda}{2}(\hat{\tau}(L)-1), \tag{9}$$

where $\hat{\sigma}_\tau(L)$ denotes the reference-profile standard deviation of the frame duration at threshold L and satisfies $\hat{\sigma}_\tau(L) \le \Delta_g$, and $\eta \ge 0$ is a robustness parameter. This robustness term discourages batches whose service times exhibit large uncertainty, thereby preventing systematic selection of high-variance batches that appear efficient only in expectation. Note that the policy only requires empirical estimates of the mean and variability of the decode latency, which can be obtained from runtime measurements, and does not require the full distribution of output token lengths.

Based on the quantized representation and the robust score defined above, we present the complete R-RDPP scheduling procedure in Algorithm 2. Compared to RDPP (Algorithm 1), the key modifications are: (i) input lengths are quantized onto the grid $\mathcal{G}_\delta$; (ii) the frame duration is estimated using the expected decode time $\hat{\tau}(L)$, provisioned at batch size k as in Algorithm 1 so that it depends on the threshold L alone, with no circular dependence on $S(L)$; and (iii) the score $\Phi_V^\eta(L)$ incorporates the robustness term $\eta\hat{\sigma}_\tau(L)$ to hedge against output-length uncertainty.

The per-decision computational complexity is $O(I_\delta N \log k)$ with $I_\delta = O(1/\delta)$, since at most I_δ thresholds are evaluated and, for each threshold, selecting up to k stations with the largest weights can be implemented using a size-k heap over N stations. This matches

the $O(I\,N \log k)$ complexity of RDPP, with I replaced by the finer grid size I_δ, offering a controllable resolution–complexity tradeoff.

We now establish the theoretical guarantees of R-RDPP. The following result characterizes its stability region.

Theorem 5.1 (Stability under Quantized Offered Slack). *Suppose Assumption 1 holds and some stationary randomized policy with thresholds in the grid $\mathcal{G}_\delta$ provides score-form offered slack $\varepsilon > 0$ at every station, uniformly over nonempty buffer compositions. Then for any $V > 0$, R-RDPP stabilizes the system whenever the robustness parameter satisfies*

$$\eta\,\Delta_g \;+\; 2\big(\Delta_g + \kappa\delta\big) \;\le\; \frac{\varepsilon\,\tau_{\min}}{2\,(\lambda^{pkt}_{\max} + \varepsilon)}, \qquad \lambda^{pkt}_{\max} := \max_i \lambda^{pkt}_i.$$

When $\Delta_g = 0$ the condition is independent of η.

Score-form offered slack extends the offered-service comparators of Theorem 4.2 to grid thresholds, with each action's service ratio taken against its own conditional mean frame duration at the current composition. The grid-sampled service bound $\mu^\star_\delta := \max_{L\in\mathcal{G}_\delta} kL/(L + f(k,L))$ satisfies $\mu^\star_\delta \ge \mu^\star - O(\delta)$, with $\mu^\star$ here maximized over the continuous input interval, so the margin consumed by quantization and uncertainty, $O(\delta + (1+\eta)\Delta_g)$, vanishes as $\delta, \Delta_g \to 0$. Remark 2 in Appendix E explains why the cap on η is imposed. We then bound R-RDPP's efficiency gap.

Theorem 5.2 (Near-Optimal Frame Efficiency under Output Uncertainty). *Under the conditions of Theorem 5.1, for an action a with conditional mean frame duration $\bar m(a)$, conditional frame-duration variance $\sigma^2_\tau(a)$, and served packet total $d(a)$, define $\psi(a) := \Lambda(\bar m(a)-1)/2 - (\bar m(a)-1)\,d(a)/\bar m(a)$ and $v(a) := \Lambda\,\sigma^2_\tau(a)/(2\bar m(a))$, and let $J^\star_\delta$ be the least worst-composition expected frame-inefficiency over grid offered-service comparators with slack ε. Then for every $V > 0$, R-RDPP's long-run duration-weighted average of $\psi + v$ satisfies*

$$\bar\psi^{\text{R-RDPP}} \;\le\; J^\star_\delta + O\Big(\frac{1}{V}\Big) + O\big(\Delta_g^2\big) + O\big((1+\eta)\Delta_g\big) + O(\delta),$$

and for every fixed V the time-average total backlog is finite, with a bound of order $O(V)$.

Theorem 5.2 extends Theorem 4.2 to output uncertainty and continuous inputs, the extra terms pricing decode variance, estimation bias with the robustness weighting, and quantization; the benchmark is necessarily causal (Remark 4), with details and constants closing Appendix F.

6 Experiments

In this section, we first conduct experiments on an NVIDIA A100 GPU to measure real-world LLM inference latency data for approximating TTFT and TPOT formulations. Then, we use NS-3 to run further experiments to show our proposed policies' great advantage over existing WiFi scheduling algorithms.

6.1 Experiments on Real-World TTFT and TPOT Measurement

6.1.1 Experimental Setup. Our experiments are conducted on an NVIDIA A100 GPU (80GB). We choose Meta-Llama-3-8B-Instruct (with 8 billion parameters) as the LLM for experiments, using a high-throughput LLM inference engine vLLM (v0.16.0) with PagedAttention for efficient KV-cache management (e.g., [16]). The model is loaded in FP16 precision with a maximum context length of 4,096 tokens and a maximum GPU memory utilization of 90%.

6.1.2 Measurement Methodology. To measure TTFT and TPOT in (1)-(2) of Section 2.1, we send concurrent streaming HTTP requests to vLLM's OpenAI-compatible API and parse Server-Sent Events (SSE) to record per-token timestamps. We set the sampling temperature to 0.0 so that each configuration produces a deterministic token sequence, ensuring that measured latency differences across grid points reflect only changes in batch size and token length rather than randomness in the decoding path. We disable prefix caching and enforce full dense matrix multiplication for every request with no KV-cache reuse, so that each measurement reflects a cold-cache inference pass. For the input workload, we construct synthetic prompts by repeating the "Quick Brown Fox" pangram to achieve mathematically precise input lengths ranging from 128 to 3,584 tokens. Since we disable prefix caching, this semantic repetition does not alter the computational complexity of the dense matrix multiplications, ensuring the measured TTFT and TPOT depend strictly on token sequence length rather than linguistic content. TTFT is then calculated as the elapsed time from sending the request to receiving the first output token as follows:

$$\text{TTFT} = t_{\text{first\_token}} - t_{\text{send}}.$$

Further, TPOT is measured as the average inter-token time after the first token as follows:

$$\text{TPOT} = \frac{t_{\text{end}} - t_{\text{first\_token}}}{n_{\text{tokens}} - 1}.$$

To simulate batch inference at batch size B, we dispatch B concurrent requests with identical prompt lengths. Each configuration is repeated 3 times and we report the mean across all individual requests. A warmup phase of 3 requests precedes all measurements to trigger the Just-In-Time (JIT) compilation of the GPU kernels, ensuring that initial compilation overhead does not artificially inflate the recorded inference latency.

6.1.3 TTFT Fitting Results. Recall from (1)-(2) in Section 2.1 that TTFT is a function of batch size and maximum input token length, TPOT is a function of batch size, maximum input token length and maximum output token length. For TTFT measurements, we fix output token length $\ell^{\text{out}} = 32$ (small enough such that TTFT dominates), sweep batch size $B \in \{1, 2, 4, 8, 16, 32\}$ and input token length $\ell^{\text{in}} \in \{128, 256, 512, 1024, 2048, 3584\}$, yielding $6 \times 6 = 36$ grid points.

We fit the 36 measured TTFT values using ordinary least squares (OLS) regression with a linear model that includes a bilinear interaction term as follows:

$$f(B, \ell^{\text{in}}_{\max}) = \beta_0 + \beta_1 B + \beta_2 \ell^{\text{in}}_{\max} + \beta_3 B \cdot \ell^{\text{in}}_{\max}. \tag{10}$$

The inclusion of the interaction term $B \cdot \ell^{\text{in}}_{\max}$ is motivated by the structure of batch prefill. The prefill computation processes all input tokens in parallel across the batch, so the marginal cost of longer inputs grows with batch size due to increased memory pressure on the KV-cache. We solve for the coefficients $\{\beta_j\}$ by minimizing $\sum_{i=1}^{36}(f(B_i, \ell^{\text{in}}_i) - \text{TTFT}_i)^2$, yielding:

$$f(B, \ell^{\text{in}}_{\max}) = 27.24 + 2.38\,B + 0.0045\,\ell^{\text{in}}_{\max} + 0.002\,B \cdot \ell^{\text{in}}_{\max}, \tag{11}$$

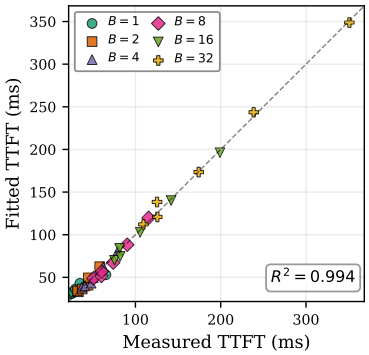

Figure 2: Parity plot of fitted versus measured TTFT across 36 grid points. Each marker shape and color represents a different batch size B. The dashed line indicates perfect prediction. The fitted model in (11) achieves $R^2 = 0.994$.

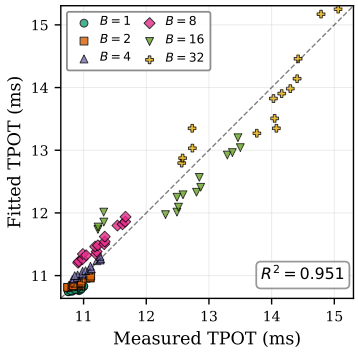

Figure 3: Parity plot of fitted versus measured TPOT across 96 grid points. Each marker shape and color represents a different batch size B. The dashed line indicates perfect prediction. The fitted model in (13) achieves $R^2 = 0.951$.

with the coefficient of determination $R^2 = 0.994$ (indicating that the fitted model explains 99.4% of the variance in the measured TTFT) and mean absolute error of 3.56 ms over the measured range of TTFT $\in [24, 351]$ ms. As shown in Figure 2, the fitted linear surface closely tracks the measured data across all batch sizes. The dominant scaling is linear in both B and $\ell^{\text{in}}_{\max}$, and the positive interaction coefficient ($\beta_3 = 0.002$) confirms that the batch-dependent prefill cost increases superlinearly when both batch size and input length are large, consistent with the max-based service cost in (1).

6.1.4 TPOT Fitting Results. For TPOT measurements, we sweep batch size $B \in \{1, 2, 4, 8, 16, 32\}$, input token length $\ell^{\text{in}} \in \{128, 512, 1024, 2048\}$, and output token length $\ell^{\text{out}} \in \{64, 128, 256, 512\}$, yielding $6 \times 4 \times 4 = 96$ grid points. We fit the 96 measured TPOT values using OLS regression with a model motivated by the memory-bandwidth bottleneck of autoregressive decoding. Each decode step reads the full KV-cache from GPU high bandwidth memory, whose size is proportional to $B \cdot (\ell^{\text{in}} + \ell^{\text{out}})$. We therefore adopt:

$$g(B, \ell^{\text{in}}_{\max}, \ell^{\text{out}}_{\max}) = \gamma_0 + \gamma_1 B + \gamma_2 B \cdot (\ell^{\text{in}}_{\max} + \ell^{\text{out}}_{\max}). \tag{12}$$

The term $\gamma_1 B$ captures per-request scheduling overhead, while $\gamma_2 B \cdot (\ell^{\text{in}}_{\max} + \ell^{\text{out}}_{\max})$ captures the KV-cache read cost that scales with both batch size and total sequence length. We solve for the coefficients $\{\gamma_j\}$ by minimizing $\sum_{i=1}^{96} (g(B_i, \ell^{\text{in}}_i, \ell^{\text{out}}_i) - \text{TPOT}_i)^2$, yielding:

$$g(B, \ell^{\text{in}}_{\max}, \ell^{\text{out}}_{\max}) = 10.68 + 0.059\, B + 3.87 \times 10^{-5}\, B \cdot (\ell^{\text{in}}_{\max} + \ell^{\text{out}}_{\max}), \tag{13}$$

with a coefficient of determination $R^2 = 0.951$ (indicating that the fitted model explains 95.1% of the variance in the measured TPOT) and a Root Mean Square Error (RMSE) of 0.27 ms over the measured range of TPOT $\in [10.7, 15.2]$ ms.

As shown in Figure 3, the fitted model closely tracks the measured data across all batch sizes. We observe that the output token length ℓ^{out} has a notably weak effect on TPOT. At fixed (B, ℓ^{in}), the spread of TPOT across all output lengths is less than 0.37 ms (e.g., at $B = 32$, $\ell^{\text{in}} = 2048$, TPOT ranges only from 14.79 to 15.15 ms). This is consistent with vLLM's paged memory management, which mitigates the KV-cache growth from output tokens. The residual variance of the fit, about 0.075 ms^2, shows hardware-induced per-token variability is negligible; the spread Δ_g in Theorem 5.2 is governed by the output-length distribution itself, exactly the uncertainty the robustness term $\eta\hat{\sigma}_\tau$ hedges.

In practice, model mismatch from fitting error or distributional shift (e.g., different tasks or hardware configurations) introduces a bounded perturbation to the frame duration estimate, structurally analogous to the output-length uncertainty already captured by $O(\Delta_g^2)$ in Theorem 5.2. The measured fitting residuals (3.56 ms for TTFT, 0.27 ms for TPOT) are small relative to typical frame durations (24–351 ms). When the operating environment changes significantly, the latency models can be recalibrated via lightweight online profiling without modifying the scheduling algorithm.

6.2 NS-3 WiFi Scheduling Evaluation with Fitted LLM Latency Models

In this subsection, we use the measured TTFT and TPOT formulations in (11) and (13) to conduct NS-3 experiments for comparing our proposed policies against existing WiFi scheduling algorithms.

We implement the WiFi-based edge AI system in NS-3.41 using the IEEE 802.11ax standard at 5 GHz with a 40 MHz channel. The AP is located at the origin and N stations (STAs) are placed uniformly on a circle of radius 10 m. The AP employs OFDMA with $k = 8$ parallel resource units, matching the batch size of the edge LLM server. Each STA generates prompts as a Bernoulli process, i.i.d. across STAs and slots with slot interval $\Delta t = 1$ ms. Prompt input lengths follow a four-class categorical distribution $\ell^{\text{in}} \in \{1, 3, 8, 15\}$ packets with probabilities $\{0.50, 0.30, 0.15, 0.05\}$, resulting in the mean of $\mathbb{E}[\ell^{\text{in}}] = 3.35$ packets. Output lengths are drawn from a truncated normal distribution with $\mu_{\text{out}} = 0.5\,\ell^{\text{in}}$ and $\sigma_{\text{out}} = 0.2\,\ell^{\text{in}}$, truncated at $\ell^{\text{out}} \geq 1$. The theory assumes bounded output support, matching deployments where the model's maximum generation length caps outputs; the simulated normal distribution is unbounded above, so the experiment is not an exact instantiation of that assumption (a draw exceeding ℓ^{in} lies 2.5 standard deviations above the mean, probability below 0.7%); it is kept exactly as in the submitted version so that all experimental results remain unchanged.

Table 2: NS-3 experiment parameters.

Parameter	Value
WiFi standard IEEE 802.11ax	5 GHz, 40 MHz
OFDMA batch size k	8
Number of STAs N	$\{5, 10, 15, 20, 25\}$
Prompt lengths ℓ^{in}	$\{1, 3, 8, 15\}$ packets
Output length	$\mathcal{N}(0.5\,\ell^{\text{in}}, (0.2\,\ell^{\text{in}})^2)$, truncated at 1
Arrival prob	$\{0.5, 0.3, 0.15, 0.05\}$ for $\ell^{\text{in}} \in \{1, 3, 8, 15\}$; $p = 0.007$
Simulation	10 s = 10,000 slots (1 ms/slot)
V, η (R-RDPP)	$V = 10$, $\eta = 1$

Each STA maintains a buffer of 1000 packets and overflow prompts are dropped. Frame duration is computed as $\tau = \ell^{\text{in}}_{\max} + f(|\mathcal{B}|, \ell^{\text{in}}_{\max}) + g(|\mathcal{B}|, \ell^{\text{in}}_{\max}, \ell^{\text{out}}_{\max}) \times (\ell^{\text{out}}_{\max} - 1)$, where $f(\cdot)$ is the fitted TTFT in (11) and $g(\cdot)$ is the fitted TPOT in (13), and each slot equals 1 ms. The executed frame duration evaluates f and g at the realized batch size, a refinement of the provisioned convention of Section 2.1 that only shortens frames; the scheduling scores of Algorithms 1–2 still use the provisioned k-based estimates, which depend on the threshold alone. We sweep STA number $N \in \{5, 10, 15, 20, 25\}$ and simulate $T = 10{,}000$ slots per run with 1,000 warmup slots. We set $V = 10$ and $\eta = 1$.

The capacity outer bound evaluates to $\mu^\star = \max_L kL/\tau(L) = 0.796$ packets/slot (attained at $L = 15$). We set the per-STA arrival probability to $p = 0.007$. In turn, the offered load relative to this outer bound, $\rho = Np\mathbb{E}[\ell^{\text{in}}]/\mu^\star$, ranges from approximately 15% at $N = 5$ to 73% at $N = 25$. The full parameter setting is summarized in Table 2.

We evaluate two system-wide metrics that directly correspond to the optimization objective in (6). The average service delay is

$$\bar{D} = \frac{\sum_{i=1}^{N} \sum_{j=1}^{K_i} (s_{i,j} - a_{i,j})}{\sum_{i=1}^{N} K_i}, \tag{14}$$

where $a_{i,j}$ and $s_{i,j}$ denote the arrival time and service start time of the j-th prompt served from station i, and K_i is the number of prompts from station i that are served after the warmup period. This metric directly measures the average waiting time experienced by a prompt in the system. The average queue length is defined as

$$\bar{Q} = \frac{1}{T - T_{\text{w}}} \sum_{t=T_{\text{w}}}^{T-1} \sum_{i=1}^{N} Q_i(t), \tag{15}$$

where $T_{\text{w}} = 1{,}000$ is the warmup period. This captures the time-averaged total number of buffered packets across all N stations after the system reaches steady state. This serves as the empirical counterpart of the theoretical objective c^π in (6). We also report the P99 tail queue length, i.e., the 99th percentile of the total queue snapshots $\sum_{i=1}^{N} Q_i(t)$ over $t > T_{\text{w}}$, which characterizes worst-case system-wide backlog relevant to interactive GenAI services. For the CDF plots, we display the empirical distribution of the total queue length $\sum_{i=1}^{N} Q_i(t)$ at $N = 15$.

Figure 4 shows the average service delay versus STA number. It indicates that our R-RDPP policy again achieves the lowest delay at every N. At $N = 5$, R-RDPP achieves roughly 350 slots versus 700–800 for benchmarks (2 times reduction). The advantage is most pronounced in the moderate-to-high load regime. At $N = 15$, R-RDPP achieves 4,000 slots versus 12,000–12,500 for benchmarks (3 times reduction). At $N = 20$, R-RDPP achieves 7,000 versus 15,000–16,000 (2.2 times reduction). Even at $N = 25$ where all policies are under heavy load (73% utilization), R-RDPP (12,000 slots) maintains a 40–45% reduction over benchmarks (19,500–21,500).

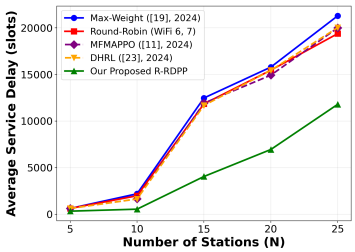

Figure 4: Average service delay versus station number N.

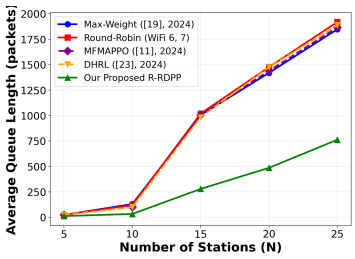

Figure 5: Average queue length versus station number N.

Figure 4 also reveals that recent RL schedulers MFMAPPO [12] and DHRL [24] for 802.11ax OFDMA can perform no better than Max-Weight and Round-Robin under GenAI workloads, with all four benchmarks clustering within 5–10% of each other. The reason is that their proportional fairness and channel-opportunistic weighting optimize along dimensions orthogonal to the batch coupling problem. When a batch mixes $\ell^{\text{in}} = 1$ with $\ell^{\text{in}} = 15$ prompts, the frame duration is forced to $\tau(15) \approx 134$ ms but only 106 of the possible 120 packets are drained, a 12% throughput loss that compounds over thousands of decisions. No amount of channel or fairness optimization can overcome this structural inefficiency, indicating that prompt-length-awareness is the critical missing ingredient in WiFi OFDMA scheduling for GenAI services.

Figure 5 shows the average queue length versus STA number. The improvement of our R-RDPP policy is still striking. At $N = 15$, R-RDPP's queue is approximately 270 packets versus 1,000–1,050 for benchmarks (3.7 times reduction). At $N = 25$, R-RDPP achieves 760 packets versus 1,850–1,920 (2.5 times reduction). Furthermore, R-RDPP's queue at $N = 25$ (760 packets) is comparable to the benchmarks' queue at $N = 15$ (1,000 packets), meaning R-RDPP effectively extends the operational range by approximately 10 additional stations before reaching the same congestion level.

Figure 6 and Table 3 show the CDF and other statistics of queue length at $N = 15$. It indicates that R-RDPP's mean queue (279 packets) is 3.5 times lower than the best benchmark (984 packets for DHRL), and its P99 (419 packets) is 4.1 times lower than benchmarks (1,717–1,737). The R-RDPP CDF reaches 1.0 at approximately 450

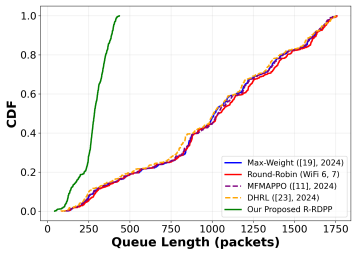

Figure 6: CDF of queue length at $N = 15$.

Table 3: Queue length statistics according to Figure 6.

Policy	Mean	Std	P99
Max-Weight [20]	1004.1	450.5	1731.0
Round-Robin (WiFi 6, 7)	1019.7	456.3	1728.0
MFMAPPO [12]	1004.8	452.8	1717.0
DHRL [24]	983.8	458.1	1737.0
Our Proposed R-RDPP	**279.4**	**85.2**	**419.0**

packets, while benchmarks extend to 1,750 packets. This reveals that R-RDPP provides substantially better tail latency guarantees under output-length uncertainty than these benchmarks.

7 Conclusion

In this work, we propose prompt-aware WiFi scheduling for GenAI services, formulating the problem as an MDP with a fundamental computational barrier. Our RDPP policy is provably stable under an explicit offered-slack condition and near-optimal in frame efficiency, and its robust extension R-RDPP handles output-length uncertainty with explicitly bounded additional costs. Experiments demonstrate significant improvements over existing WiFi schedulers.

An interesting future direction is to extend the framework to continuous batching, where the renewal structure would need to be replaced by a more general semi-regenerative analysis.

References

[1] Boris Bellalta. 2016. IEEE 802.11ax: High-efficiency WLANs. *IEEE Wireless Communications* 23, 1 (2016), 38–46.

[2] Dimitri P Bertsekas. 2012. *Dynamic Programming and Optimal Control* (4th ed.). Vol. 2. Athena Scientific.

[3] Peter Brucker, Andrei Gladky, Han Hoogeveen, Mikhail Y. Kovalyov, Chris N. Potts, Thomas Tautenhahn, and Steef L. van de Velde. 1998. Scheduling a batching machine. *Journal of Scheduling* 1, 1 (1998), 31–54.

[4] Runze Cheng, Yao Sun, Dusit Niyato, Lan Zhang, Lei Zhang, and Muhammad Ali Imran. 2024. A wireless AI-generated content (AIGC) provisioning framework empowered by semantic communication. *IEEE Transactions on Mobile Computing* 24, 3 (2024), 2137–2150.

[5] Jim G Dai. 1995. On positive Harris recurrence of multiclass queueing networks: a unified approach via fluid limit models. *The Annals of Applied Probability* 5, 1 (1995), 49–77.

[6] Atilla Eryilmaz and R Srikant. 2007. Fair resource allocation in wireless networks using queue-length-based scheduling and congestion control. *IEEE/ACM Transactions on Networking* 15, 6 (2007), 1333–1344.

[7] Agrawal et al. 2024. Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX symposium on operating systems design and implementation (OSDI 24)*. 117–134.

[8] John W. Fowler and Lars Mönch. 2022. A survey of scheduling with parallel batch (p-batch) processing. *European Journal of Operational Research* 298, 1 (2022), 1–24.

[9] Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. 2024. Efficient llm scheduling by learning to rank. *Advances in Neural Information Processing Systems* 37 (2024), 59006–59029.

[10] Leonidas Georgiadis, Michael J Neely, and Leandros Tassiulas. 2006. *Resource allocation and cross-layer control in wireless networks*. Now Publishers Inc.

[11] Cem Güldoğan et al. 2024. Multi-bin batching for increasing LLM inference throughput. *arXiv preprint arXiv:2412.04504* (2024).

[12] Mingqi Han, Xinghua Sun, Wen Zhan, Yayu Gao, and Yuan Jiang. 2024. Multi-agent reinforcement learning based uplink OFDMA for IEEE 802.11 ax networks. *IEEE Transactions on Wireless Communications* 23, 8 (2024), 8868–8882.

[13] S. Hao and L. Duan. 2026. Prompt-Aware WiFi Scheduling for Generative AI Services. Technical Report. https://shuganghao.github.io/files/hao2026WiFi.pdf

[14] Sunghyun Jin, Serae Kim, Sangtae Ha, and Kyunghan Lee. 2025. End-to-End Coordination of RAN and Edge Server for Latency-Critical Inference Serving over Cellular Networks. *Proceedings of the ACM on Networking* 3, CoNEXT4 (2025), 1–23.

[15] Minsu Kim, Pinyarash Pinyoanuntapong, Bongho Kim, Walid Saad, and Doru Calin. 2025. Edge vs cloud: How do we balance cost, latency, and quality for large language models over 5G networks?. In *2025 IEEE Wireless Communications and Networking Conference (WCNC)*. IEEE, 1–6.

[16] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.

[17] Zheng Lin, Guanqiao Qu, Qiyuan Chen, Xianhao Chen, Zhe Chen, and Kaibin Huang. 2025. Pushing large language models to the 6g edge: Vision, challenges, and opportunities. *IEEE Communications Magazine* 63, 9 (2025), 52–59.

[18] Xiaoqian Liu, Yuhan Dong, Yiqing Li, Yousi Lin, and Ming Gan. 2025. IEEE 802.11 be Wi-Fi 7: Feature summary and performance evaluation. *IEEE Communications Standards Magazine* (2025).

[19] Álvaro López-Raventós and Boris Bellalta. 2022. Multi-link operation in IEEE 802.11 be WLANs. *IEEE Wireless Communications* 29, 4 (2022), 94–100.

[20] Xiaoping Ma, Xiangdong Jia, Yue Li, and Kailai Xue. 2024. Minimizing age of information in wireless powered communication network based on short packet communication. In *2024 IEEE 100th Vehicular Technology Conference (VTC2024-Fall)*. IEEE, 1–5.

[21] Davide Magrin, Stefano Avallone, Sumit Roy, and Michele Zorzi. 2023. Performance evaluation of 802.11 ax OFDMA through theoretical analysis and simulations. *IEEE Transactions on Wireless Communications* 22, 8 (2023), 5070–5083.

[22] Sean P. Meyn and Richard L. Tweedie. 2012. *Markov Chains and Stochastic Stability*. Springer Science & Business Media.

[23] Michael Neely. 2010. *Stochastic network optimization with application to communication and queueing systems*. Morgan & Claypool Publishers.

[24] Hyeonho Noh, Harim Lee, and Hyun Jong Yang. 2024. Joint optimization on uplink OFDMA and MU-MIMO for IEEE 802.11 ax: Deep hierarchical reinforcement learning approach. *IEEE Communications Letters* 28, 8 (2024), 1800–1804.

[25] NVIDIA. 2024. NVIDIA NIM Benchmarks for LLM Inference. https://docs.nvidia.com/nim/benchmarking/llm/latest/performance.html.

[26] Michele Polese, Leonardo Bonati, Salvatore D'oro, Stefano Basagni, and Tommaso Melodia. 2023. Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges. *IEEE Communications Surveys & Tutorials* 25, 2 (2023), 1376–1411.

[27] Yibin Shen and Zili Meng. 2026. Law: Towards consistent low latency in 802.11 home networks. In *Proc. USENIX NSDI*.

[28] Leandros Tassiulas and Anthony Ephremides. 1992. Stability properties of constrained queueing systems and scheduling policies for maximum throughput in multihop radio networks. *IEEE Trans. Automat. Control* 37, 12 (1992), 1936–1948.

[29] Jingyu Wang, Xuming Fang, Dusit Niyato, and Tie Liu. 2024. Next-generation wi-fi networks with generative ai: Design and insights. *arXiv preprint arXiv:2408.04835* (2024).

[30] Jinghang Xu, Kun Guo, Wei Teng, Chenxi Liu, and Wei Feng. 2026. Batch denoising for AIGC service provisioning in wireless edge networks. *IEEE Transactions on Vehicular Technology* (2026).

[31] Chien-Sheng Yang, Yu-Jen Ku, Yuan-Yao Lou, Nathan Tenny, and Alex C-C Hsu. 2025. 6G EdgeAI: performance evaluation and analysis. *arXiv preprint arXiv:2504.16529* (2025).

[32] Yuqing Yang, Lei Jiao, and Yuedong Xu. 2024. A queueing theoretic perspective on low-latency llm inference with variable token length. In *2024 22nd International Symposium on Modeling and Optimization in Mobile, Ad Hoc, and Wireless Networks (WiOpt)*. IEEE, 273–280.

[33] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, et al. 2022. Orca: A distributed serving system for transformer-based generative models. In *USENIX OSDI*. 521–538.

[34] J. J. Yuan, Y. X. Lin, T. C. E. Cheng, and C. T. Ng. 2007. Single machine serial-batching scheduling problem with a common batch size to minimize total weighted completion time. *International Journal of Production Economics* 105, 2 (2007), 402–406.

[35] Xinyuan Zhang, Jiangtian Nie, Yudong Huang, Gaochang Xie, Zehui Xiong, Jiang Liu, Dusit Niyato, and Xuemin Shen. 2024. Beyond the cloud: Edge inference for generative large language models in wireless networks. *IEEE Transactions on Wireless Communications* 24, 1 (2024), 643–658.

A Proof of Lemma 2.3

Lemma 2.3 states that max-weight and round-robin can fail on stabilizable systems. Part (a) constructs a max-weight counterexample, stabilized by an explicit stationary policy, for every target rate $\Lambda_0 > 0$; Part (b) constructs a round-robin counterexample for every $\Lambda_0 \in (0, 1)$.

A.1 Part (a): Instability of Max-Weight Scheduling

Proof. Fix any target arrival rate $\Lambda_0 > 0$. We construct a system instance whose total packet arrival rate equals Λ_0, exhibit an explicit stationary policy that stabilizes it (so that the instance is stabilizable), and then prove that max-weight (longest-queue-first, LQF) scheduling is unstable on the same instance.

Instance construction. Define the TTFT function

$$f(B, \ell^{\text{in}}_{\max}) \;=\; (\ell^{\text{in}}_{\max})^2,$$

which is nondecreasing in both arguments, and set $g \equiv 0$. We use two prompt types $\ell^{\text{in}} \in \{L^{\text{in}}_1, L^{\text{in}}_2\} = \{1, M\}$, where M is an integer parameter fixed below. A frame whose transmitted prompts are all short has duration $\tau(1) = 1 + 1 = 2$, while a frame that transmits at least one long prompt has duration

$$T_L \;:=\; \tau(M) \;=\; M + M^2.$$

The outer bound of Lemma 2.3 evaluates to $\mu^\star = \max\{k/2,\ kM/T_L\} = k/2$, attained at $\ell = 1$.

Let $\beta := \lfloor \Lambda_0 \rfloor + 1$, so that $\Lambda_0 < \beta$, and let M be any integer with

$$M \;\geq\; M_0 \;:=\; \max\left\{12,\ \left\lceil \frac{4(2\beta+1)}{3\Lambda_0} \right\rceil + 1\right\}.$$

Choose the system parameters

$$N \;=\; k \;=\; \beta M, \qquad \varepsilon \;=\; \frac{1}{2k}, \qquad p \;=\; \frac{2\Lambda_0}{2k + M - 1}.$$

In each slot, each station generates a prompt with probability p, which is long (length M) with probability ε and short (length 1) with probability $1-\varepsilon$, independently across stations and slots; that is, $\alpha_1 = p(1-\varepsilon)$ and $\alpha_2 = p\varepsilon$. The mean prompt length is $\bar{\ell} = 1 - \varepsilon + \varepsilon M = 1 + (M-1)/(2k)$, so the total packet arrival rate is

$$\Lambda \;=\; Np\,\bar{\ell} \;=\; p\left(k + \frac{M-1}{2}\right) \;=\; \Lambda_0$$

exactly. Three elementary consequences of these choices are used below: (i) $k\varepsilon = 1/2$; (ii) $p \leq \Lambda_0/k < 1/M \leq 1/12$, since $\Lambda_0 < \beta$; and (iii) $\Lambda_0 < \beta M/2 = \mu^\star$. The system-wide arrival rate of long *prompts* is $\nu := Np\varepsilon = p/2$ per slot.

Since $N = k$, max-weight selects every nonempty station in each frame (empty stations carry zero weight, so including or excluding them is immaterial). Being length-oblivious, it transmits each scheduled station's head-of-line (HOL) prompt in FIFO order, as in Part (b). A frame is *short* (duration 2) if all transmitted prompts are short and *long* (duration T_L) if at least one is long; when all stations are empty the AP idles for one slot.

Stabilizability verification. Let $c := \lceil 3pT_L \rceil$ and $T_c := 2c + T_L$. Consider the stationary randomized policy $\Pi^\star$: at each frame epoch, independently of the past, with probability $\theta := 1/(c+1)$ run a *pure-long frame* (every station holding at least one long prompt transmits its oldest long prompt; duration T_L), and with probability $1-\theta$ run a *pure-short frame* (every station holding at least one short prompt transmits its oldest short prompt; duration 2). If no eligible prompt exists the AP idles for the corresponding duration. The policy schedules at most $k = N$ stations and one prompt per scheduled station, and it never mixes types, so the stated durations follow from $\tau(\cdot)$.

We first record two load bounds. Since $p < 1/12$ and $T_L \geq 4$,

$$2c \;\leq\; 6pT_L + 2 \;\leq\; \tfrac{1}{2}T_L + \tfrac{1}{2}T_L \;=\; T_L, \qquad \text{hence} \qquad T_c \;\leq\; 2T_L.$$

Short prompts: per station, short prompts arrive at rate $p(1-\varepsilon)$ per slot, and

$$p(1-\varepsilon)\,T_c \;\leq\; 2pT_L \;\leq\; \tfrac{2}{3}\,c. \tag{16}$$

Long prompts: per station, long prompts arrive at rate $p\varepsilon$ per slot. Since $p = 2\Lambda_0/(2k+M-1)$ with $2k+M-1 = (2\beta+1)M - 1 \geq \frac{35}{36}(2\beta+1)M$ (as $(2\beta+1)M \geq 36$ for $\beta \geq 1$, $M \geq 12$), and using $M+1 \leq \frac{13}{12}M$, $T_c \leq 2T_L$, $2\beta+1 \geq 3$, and $\Lambda_0 < \beta$,

$$p\varepsilon\,T_c \;\leq\; \frac{p\,T_L}{k} \;=\; \frac{2\Lambda_0(M+1)}{\beta\big((2\beta+1)M-1\big)} \;\leq\; \frac{78}{35}\cdot\frac{\Lambda_0}{\beta(2\beta+1)} \;\leq\; \frac{26}{35} \;<\; \frac{4}{5}. \tag{17}$$

Under $\Pi^\star$ the service offered to each station is driven only by the exogenous i.i.d. frame-type sequence, so the stations decouple. Sample station i's buffer at frame epochs and let D denote a frame duration, with $\mathbb{E}[D] = 2(1-\theta) + \theta T_L = T_c/(c+1)$. Its short count S_i receives one service with probability $1-\theta = c/(c+1)$ whenever $S_i > 0$, against mean per-frame arrivals $p(1-\varepsilon)\mathbb{E}[D]$, giving per-frame drift at most $[\,p(1-\varepsilon)T_c - c\,]/(c+1) \leq -c/(3(c+1))$ by (16). Its long count L_i receives one service with probability θ whenever $L_i > 0$, against mean arrivals $p\varepsilon\,\mathbb{E}[D]$, giving drift at most $[\,p\varepsilon T_c - 1\,]/(c+1) \leq -1/(5(c+1))$ by (17).

Both sampled chains have increments with uniformly bounded exponential moments (arrivals over at most T_L slots) and strictly negative drift outside a finite set, so by the standard Foster–Lyapunov argument with a quadratic Lyapunov function (e.g., [10], [23]) each is positive

recurrent with finite stationary mean; since frame durations are bounded by T_L, the slot-sampled backlog $\sum_i (S_i + ML_i)$ also has finite long-run average. Hence $\bar{Q}^{\Pi^\star} < \infty$: the stationary randomized policy $\Pi^\star$ serves every station with positive slack, so the instance is stabilizable.

Instability of max-weight. Suppose for contradiction that LQF is stable on this instance, i.e., $\bar{Q}^{\mathrm{LQF}} < \infty$.

Step 1: Mean rate stability. Since $\bar{Q}^{\mathrm{LQF}} = \limsup_T \frac{1}{T} \sum_{t<T} \mathbb{E}[Q_{\mathrm{tot}}(t)] < \infty$, we cannot have $\mathbb{E}[Q_{\mathrm{tot}}(t)] \geq \delta t$ for all large t and any $\delta > 0$; hence there is a sequence $T_j \to \infty$ with $\mathbb{E}[Q_{\mathrm{tot}}(T_j)]/T_j \to 0$. Every buffered prompt contributes at least one packet, and every buffered long prompt contributes M packets. Therefore, along T_j: (i) for each station i, the expected number of prompts transmitted by station i up to T_j equals its expected arrivals minus its expected buffered prompts, hence is at least $pT_j - o(T_j)$; and (ii) the expected number of long prompts transmitted system-wide up to T_j is at least $\nu T_j - o(T_j)$.

Step 2: Frame accounting. Each scheduled station transmits one prompt per frame, so the number $n(T)$ of frames started by time T satisfies $\mathbb{E}[n(T_j)] \geq pT_j - o(T_j)$ by Step 1(i). Frames do not overlap, frames have duration 2 or T_L, and at most one frame started by T ends after T; writing $n_L(T)$ for the number of long frames started by T,

$$2\, n(T) + (T_L - 2)\, n_L(T) \;\leq\; T + T_L. \tag{18}$$

Step 3: LQF cannot batch long prompts. Prompt types are i.i.d. marks, drawn independently of the arrival times. On this instance, the entire LQF schedule up to the start of any frame t is a deterministic function of the arrival times and of the types of *previously transmitted* prompts only: the selection is exhaustive over nonempty stations (a function of arrival times and past frame durations), the transmission order within each station is FIFO, and past frame durations depend only on transmitted types. Types of prompts still waiting in the buffers influence nothing that has happened.

Consequently, conditional on the history $\mathcal{F}_t$ available at the start of frame t, the HOL types of the $m_t \geq 1$ nonempty stations are i.i.d. Bernoulli(ε), so the number J_t of long prompts transmitted in frame t satisfies $J_t \mid \mathcal{F}_t \sim \mathrm{Binomial}(m_t, \varepsilon)$. Hence $\mathbb{E}[J_t \mid \mathcal{F}_t] = m_t\varepsilon$, and by Bonferroni and $(k-1)\varepsilon < k\varepsilon = 1/2$,

$$\Pr(J_t \geq 1 \mid \mathcal{F}_t) \;=\; 1 - (1-\varepsilon)^{m_t} \;\geq\; m_t\varepsilon\Big(1 - \frac{(m_t - 1)\varepsilon}{2}\Big) \;\geq\; \frac{3}{4}\, m_t\varepsilon \;=\; \frac{3}{4}\, \mathbb{E}[J_t \mid \mathcal{F}_t].$$

Thus $\mathbb{E}\big[\mathbf{1}\{J_t \geq 1\} - \frac{3}{4} J_t \,\big|\, \mathcal{F}_t\big] \geq 0$ for every frame. Since frames last at least two slots, at most $\lceil T/2 \rceil + 1$ frames start by time T, the event that frame t starts by time T is $\mathcal{F}_t$-measurable, and every transmitted long prompt belongs to some started frame; summing the conditional inequality over frames and invoking Step 1(ii),

$$\mathbb{E}[n_L(T_j)] \;\geq\; \frac{3}{4}\, \mathbb{E}\Big[\, \textstyle\sum_t J_t \,\Big] \;\geq\; \tfrac{3}{4}\big(\nu T_j - o(T_j)\big). \tag{19}$$

In words, each long frame under LQF transmits at most 4/3 long prompts on average: LQF transmits a long prompt in the first frame after it reaches a HOL and has no mechanism to defer it, so long prompts almost never share the frame whose duration they inflate, whereas $\Pi^\star$ amortizes one duration-T_L frame over up to k long prompts.

Step 4: Contradiction. Taking expectations in (18) at $T = T_j$, inserting the bounds of Step 2 and (19), dividing by T_j, and letting $j \to \infty$ yields the necessary condition

$$1 \;\geq\; 2p + \frac{3}{4}\,(T_L - 2)\,\nu \;=\; 2p + \frac{3}{8}\, p\,(M^2 + M - 2),$$

using $\nu = p/2$. But $M^2 + M - 2 \geq M^2$ for $M \geq 2$ and $p \geq 2\Lambda_0/((2\beta+1)M)$, so

$$2p + \frac{3}{8}\, p\,(M^2 + M - 2) \;\geq\; \frac{3}{8} \cdot \frac{2\Lambda_0}{(2\beta+1)M} \cdot M^2 \;=\; \frac{3\Lambda_0 M}{4(2\beta+1)} \;>\; 1,$$

where the last inequality holds because $M \geq M_0 > 4(2\beta+1)/(3\Lambda_0)$. This contradiction shows $\bar{Q}^{\mathrm{LQF}} = \infty$. Hence max-weight scheduling is unstable on a stabilizable instance (whose total arrival rate satisfies $\Lambda_0 < \mu^\star = k/2$), which proves Part (a). As a concrete example, $\Lambda_0 = 2$ gives $\beta = 3$, $M = 12$, $N = k = 36$, $\varepsilon = 1/72$, and $p = 4/83$, for which the right-hand side of the necessary condition evaluates to $2.88 > 1$, so LQF would need 2.88 time units of service per unit of time. □

Remark. The construction takes $N = k$, under which the max-weight selection is exhaustive; this choice isolates the source of instability and admits an exact analysis. The failure exhibited here is not caused by the backlog-ranking rule but by the length-oblivious FIFO service that max-weight shares with many schedulers: it transmits a long prompt as soon as that prompt reaches a head-of-line position, so nearly every long prompt triggers its own duration-T_L frame, and the max-based inference delay is paid once per long prompt rather than once per batch of k long prompts. The stabilizing policy $\Pi^\star$ instead separates types and amortizes the long-frame cost, which is exactly the prompt-aware batching behavior developed in Sections 3–4.

A.2 Part (b): Instability of Round-Robin Scheduling

Proof. Fix any target arrival rate $\Lambda_0 \in (0,1)$. As in Part (a), we construct a system instance whose total packet arrival rate equals Λ_0, verify that the instance is stabilizable, and then prove that round-robin (RR) scheduling is unstable on the same instance.

Instance construction. Let $k \geq 2$ be an integer parameter fixed below and define the TTFT function

$$f(B, \ell^{\text{in}}_{\max}) \;=\; B\,\ell^{\text{in}}_{\max},$$

which is nondecreasing in both arguments, with $g \equiv 0$. Under the provisioned batch size $B = k$ of Section 2.1, a frame whose maximum transmitted length is ℓ has duration $\tau(\ell) = \ell + k\ell = (1+k)\ell$, so the normalized service rate $k\ell/\tau(\ell) = k/(1+k)$ is the same for every length and $\mu^\star = k/(1+k)$.

Use two prompt types $\ell^{\text{in}} \in \{1, M\}$ with an integer $M \geq 2$. Set

$$N = \max\{2k,\ \lceil 10k^2/\Lambda_0 \rceil\}, \qquad \varepsilon = \frac{1}{3M}, \qquad \bar{\ell} = \varepsilon M + (1-\varepsilon) = \frac{4M-1}{3M}, \qquad p = \frac{\Lambda_0}{N\bar{\ell}}.$$

In each slot, each station independently generates a prompt with probability p; conditional on arrival, the prompt is long (length M) with probability ε and short (length 1) otherwise, i.i.d. across stations and slots. The total packet arrival rate is $\Lambda = Np\bar{\ell} = \Lambda_0$ exactly, and the system-wide arrival rate of long *prompts* is

$$\nu \;:=\; Np\varepsilon \;=\; \frac{\Lambda_0}{3M\bar{\ell}} \;=\; \frac{\Lambda_0}{4M-1} \quad \text{per slot.}$$

RR policy. RR cycles through the N stations in a fixed cyclic order: at each frame epoch it scans forward from its pointer and selects the first $m_t := \min\{k,\ \#\{\text{nonempty stations}\}\}$ nonempty stations, each of which transmits its head-of-line (HOL) prompt under FIFO, after which the pointer advances past the last selected station; when every station is empty the AP idles for one slot. RR is length-oblivious: its selections depend on buffer occupancies only, never on the lengths of untransmitted prompts. A frame is *long* (duration $(1+k)M$) if at least one transmitted prompt is long and *short* (duration $1+k$) otherwise, the policy implicitly declaring the realized maximum per Section 3.1.

Stabilizability verification. Since the normalized service rate $k\ell/\tau(\ell) = k/(1+k)$ is independent of ℓ, the type-separated policies of Remark 1 offer $k/(1+k)$ packets per slot regardless of which length class they serve; choosing the class weights proportional to the per-class packet loads therefore offers every station positive slack whenever $\Lambda_0 < \mu^\star = k/(1+k)$, which the choice of k below guarantees. Hence the instance is stabilizable.

Instability of RR. Suppose for contradiction that RR is stable on this instance, i.e., $\bar{Q}^{\text{RR}} < \infty$.

Step 1: Mean rate stability. Since $\bar{Q}^{\text{RR}} = \limsup_T \frac{1}{T}\sum_{t<T}\mathbb{E}[Q_{\text{tot}}(t)] < \infty$, there is a sequence $T_j \to \infty$ with $\mathbb{E}[Q_{\text{tot}}(T_j)]/T_j \to 0$. Every buffered long prompt contributes $M \geq 1$ packets, so the expected number of long prompts still buffered at T_j is $o(T_j)$, and the expected number of long prompts transmitted system-wide up to T_j is at least $\nu T_j - o(T_j)$.

Step 2: Deferred decisions. Prompt types are i.i.d. marks, drawn independently of the arrival times. Under RR, the entire schedule up to the start of any frame t is a deterministic function of the arrival times and of the types of *previously transmitted* prompts only. Indeed, by induction over frames: which stations are nonempty at a frame epoch is determined by per-station arrival counts and past selections, since each selected station loses exactly one prompt regardless of its type; the selection itself is the deterministic pointer-order scan over nonempty stations; the within-station order is FIFO; and each past frame's duration depends only on the types it transmitted. Types of prompts still waiting in the buffers influence nothing that has happened. Consequently, conditional on the history $\mathcal{F}_t$ available at the start of frame t, the HOL types of the $m_t \geq 1$ selected stations are i.i.d. Bernoulli(ε), so the number J_t of long prompts transmitted in frame t satisfies $J_t \mid \mathcal{F}_t \sim \text{Binomial}(m_t, \varepsilon)$ with $\mathbb{E}[J_t \mid \mathcal{F}_t] = m_t\varepsilon$.

Step 3: RR cannot batch long prompts. Since $m_t \leq k$ and $M \geq k-1$ (guaranteed by $M = k^2$ below), we have $(m_t - 1)\varepsilon \leq (k-1)/(3M) \leq 1/3$, so by Bonferroni,

$$\Pr(J_t \geq 1 \mid \mathcal{F}_t) \;=\; 1 - (1-\varepsilon)^{m_t} \;\geq\; m_t\varepsilon\Big(1 - \frac{(m_t-1)\varepsilon}{2}\Big) \;\geq\; \frac{5}{6}\,m_t\varepsilon \;=\; \frac{5}{6}\,\mathbb{E}[J_t \mid \mathcal{F}_t],$$

hence $\mathbb{E}\big[\mathbf{1}\{J_t \geq 1\} - \frac{5}{6}J_t \,\big|\, \mathcal{F}_t\big] \geq 0$ for every frame. Frames last at least $1+k$ slots, so at most $\lceil T/(1+k)\rceil + 1$ frames start by time T; the event that frame t starts by time T is $\mathcal{F}_t$-measurable; and every transmitted long prompt belongs to some started frame. Writing $n_L(T)$ for the number of long frames started by time T and summing the conditional inequality over frames, Step 1 gives

$$\mathbb{E}[n_L(T_j)] \;\geq\; \frac{5}{6}\,\mathbb{E}\Big[\textstyle\sum_t J_t\Big] \;\geq\; \tfrac{5}{6}\big(\nu T_j - o(T_j)\big). \tag{20}$$

In words, RR transmits at most 6/5 long prompts per long frame on average: being length-oblivious, it transmits each long prompt in the first frame after that prompt reaches a HOL position and has no mechanism to defer it, so long prompts almost never share the duration-$(1+k)M$ frame they inflate, whereas a type-separated policy amortizes one long frame over up to k long prompts.

Step 4: Frame accounting and contradiction. Frames occupy disjoint time intervals and at most one frame started by T ends after T, so the durations of started frames satisfy $(1+k)M\, n_L(T) \le T + (1+k)M$. Taking expectations at $T = T_j$, inserting (20), dividing by T_j, and letting $j \to \infty$ yields the necessary condition

$$1 \;\ge\; \frac{5}{6}\,(1+k)M\,\nu \;=\; \frac{5}{6}\cdot\frac{(1+k)M\,\Lambda_0}{4M-1} \;>\; \frac{5}{24}\,(1+k)\,\Lambda_0,$$

using $M/(4M-1) > 1/4$. For any $\Lambda_0 \in (0,1)$, choose

$$k^\dagger := \max\left\{\left\lceil \frac{6}{\Lambda_0}\right\rceil + 1,\ \left\lceil \frac{\Lambda_0}{1-\Lambda_0}\right\rceil + 1,\ 18\right\}, \qquad M^\dagger := (k^\dagger)^2.$$

Then $\mu^\star = k^\dagger/(1+k^\dagger) > \Lambda_0$, so the instance is stabilizable by the verification above, while

$$\frac{5}{24}\,(1+k^\dagger)\,\Lambda_0 \;\ge\; \frac{5}{24}\Big(\frac{6}{\Lambda_0}+2\Big)\Lambda_0 \;=\; \frac{5}{4}+\frac{5\Lambda_0}{12} \;>\; 1,$$

contradicting the necessary condition. Hence $\bar{Q}^{\mathrm{RR}} = \infty$: round-robin is unstable on a stabilizable instance, which proves Part (b). As a concrete example, $\Lambda_0 = 1/2$ gives $k^\dagger = 18$, $M^\dagger = 324$, and $\frac{5}{6}(1+k^\dagger)M^\dagger\nu = 1.98 > 1$, so RR would need 1.98 time units of long-frame service per unit of time. □

B Complexity Status Behind Lemma 3.1

We make the two barriers in Lemma 3.1 precise. The exponential state space is immediate from the definition of $\mathcal{B}(t)$: with N stations and I prompt types, already the states reachable from buffers holding at most one prompt per station number $(I+1)^N$. We therefore focus on the embedded clearing problem, first exhibiting the embedding and then discussing its complexity status.

Embedding: Constructing the MDP instance. Given prompt lengths $p_1, \dots, p_n \in \mathbb{Z}_{>0}$ and a batch capacity b, construct the following MDP instance:

- *Stations:* $N = n$, one per job.
- *Prompt types:* $I = |\{p_1, \dots, p_n\}|$ distinct lengths; $\{L_1^{\mathrm{in}}, \dots, L_I^{\mathrm{in}}\} = \{p_1, \dots, p_n\}$.
- *Initial state:* Station i has exactly one prompt of length p_i:

$$Q_i(0) = p_i, \qquad n_{i,j}(0) = \mathbf{1}\{L_j^{\mathrm{in}} = p_i\}, \qquad r(0) = 0.$$

- *No future arrivals:* $\Pr(\ell_i^{\mathrm{in}}(t) = 0) = 1$ for all i and $t \ge 1$.
- *Parameters:* $k = b$ (batch capacity), $f(m,\ell) \equiv 0$ (so $\tau = \ell_{\max}^{\mathrm{sel}}$), and $g \equiv 0$.

This construction is polynomial in the input size.

Policy correspondence. Any feasible MDP policy induces a valid batch schedule and vice versa: an MDP scheduling action at slot t that selects stations $S_t^{\mathrm{sched}} \subseteq [N]$ with $|S_t^{\mathrm{sched}}| \le k$ corresponds to forming a batch of the corresponding jobs starting at time t.

Objective equivalence. Consider station i (job i) first scheduled in a batch at slot S_i. Since $f \equiv 0$, the batch duration is $\tau = \max_{j\in\text{batch}} p_j$, and station i's prompt departs at the end of slot $S_i + \tau - 1$. Station i's contribution to the MDP cost is

$$\sum_{t=0}^{\infty} Q_i(t) = p_i \cdot (S_i + 1),$$

since $Q_i(t) = p_i$ for $t = 0, 1, \dots, S_i$ (present before and during the scheduling slot) and $Q_i(t) = 0$ for $t > S_i$ (departed). The total MDP cost is therefore

$$\sum_{t=0}^{\infty}\sum_{i=1}^{n} Q_i(t) = \sum_{i=1}^{n} p_i(S_i+1) = \sum_{i=1}^{n} p_i S_i + \sum_{i=1}^{n} p_i.$$

Since $\sum_i p_i$ is a constant independent of the policy, minimizing the MDP cost is equivalent to minimizing $\sum_i p_i S_i$.

Complexity status. The embedding shows that exact policy computation for the MDP contains the clearing problem of Lemma 3.1, a bounded parallel-batch scheduling problem in which each prompt's weight equals its own length. Three features place this problem outside the reach of the classified batching literature. First, in the classical bounded problem $1 \mid p\text{-batch},\ b < n \mid \sum C_j$ the jobs are unweighted, and its complexity was listed as unresolved [3, 8]; we are unaware of a complexity classification for the proportional-weight, batch-start-cost variant embedded here. Second, the cost accrues at batch start rather than batch completion, which removes the own-batch term present in the known strong NP-hardness result for serial batching with proportional weights [34], so that result does not directly transfer. Third, batch occupancy depends on the selected maximum length only, not on the sum of the selected lengths, which excludes a direct embedding of sum-based batching models.

We therefore refrain from asserting NP-hardness; to the best of our knowledge, no complexity classification is available for the embedded clearing problem.

What can be said rigorously is that the optimal clearing schedule has no simple structure: for lengths $\{3, 8, 8, 10\}$ with $k = 2$ and $f \equiv 0$, the unique optimal partition is $\{8, 8\}, \{10, 3\}$ with cost 104, strictly better than every sorted-consecutive partition (best value 110), so neither ascending nor descending sorted-consecutive batching is always optimal, and no polynomial-time exact algorithm is known. Together with the exponential state space, this justifies resorting to the low-complexity policy of Section 4, whose guarantees are established directly and do not rely on solving the ACOE.

Transfer to the average-cost criterion. The no-arrival construction above is a finite clearing embedding and does not by itself establish hardness of the long-run average-cost MDP, whose value is zero once all prompts are cleared. Under recurrent arrivals, analogous clearing configurations reappear with positive probability, illustrating that the same batch-partitioning structure can arise during average-cost operation. We use this observation only as motivation, not as a complexity reduction or hardness claim.

C Proof of Lemma 4.1

Before proving the main stability result, we establish several intermediate lemmas that characterize the per-frame dynamics.

Lemma C.1 (Per-Frame Queue Evolution). *Let T_n denote the n-th renewal epoch (decision epoch when $r(T_n) = 0$) and $\tau_n := T_{n+1} - T_n$ be the frame duration. At a renewal epoch T_n, let the RDPP policy select threshold L_n, batch $S_n := S(L_n)$, and prompt lengths $\{\ell_i(L_n)\}_{i\in S_n}$. Then the post-arrival queue length at the next renewal epoch satisfies, for each station i:*

$$Q_i^+(T_{n+1}) = Q_i^+(T_n) - \mathbf{1}\{i \in S_n\}\,\ell_i(L_n) + \sum_{u=1}^{\tau_n} \ell_i^{in}(T_n + u), \tag{21}$$

where $\ell_i^{in}(T_n + u)$ denotes the packet arrival at station i in slot $T_n + u$.

Proof. Service occurs at the end of slot T_n, removing $\ell_i(L_n)$ packets from station i if $i \in S_n$. During the frame (slots $T_n+1, \dots, T_n+\tau_n = T_{n+1}$), no further scheduling occurs ($r(t) > 0$) and only arrivals accumulate. Summing arrivals over the frame yields (21). □

Lemma C.2 (Per-Frame Lyapunov Drift Bound). *Define the quadratic Lyapunov function $\mathcal{L}(\boldsymbol{Q}) := \frac{1}{2}\sum_{i=1}^N (Q_i^+)^2$. Let $\mathcal{F}_n$ denote the filtration at epoch T_n. The conditional frame drift satisfies*

$$\mathbb{E}\big[\mathcal{L}(\boldsymbol{Q}^+(T_{n+1})) - \mathcal{L}(\boldsymbol{Q}^+(T_n))\,\big|\,\mathcal{F}_n\big] \le \tilde{B}_0 + \sum_{i=1}^N Q_i^+(T_n)\Big(\lambda_i^{pkt}\,\tau_n - \mathbf{1}\{i \in S_n\}\,\ell_i(L_n)\Big), \tag{22}$$

where $\lambda_i^{pkt} := \mathbb{E}[\ell_i^{in}(t)]$, $\tau_n = \tau(L_n)$, and $\tilde{B}_0 := \frac{1}{2}\sum_{i=1}^N \Big[\bar{\tau}\,\mathbb{E}[(\ell_i^{in})^2] + \bar{\tau}^2(\lambda_i^{pkt})^2 + (L_I^{in})^2\Big] < \infty$ with $\bar{\tau} := \max_L \tau(L)$.

Proof. Define $A_i := \sum_{u=1}^{\tau_n} \ell_i^{\text{in}}(T_n+u)$ and $D_i := \mathbf{1}\{i \in S_n\}\,\ell_i(L_n)$. By Lemma C.1, $Q_i^+(T_{n+1}) = Q_i^+(T_n) + A_i - D_i$. Using $(a+b)^2 - a^2 = 2ab + b^2$:

$$\begin{aligned}&\mathcal{L}(\boldsymbol{Q}^+(T_{n+1})) - \mathcal{L}(\boldsymbol{Q}^+(T_n))\\ &= \sum_{i=1}^N Q_i^+(T_n)(A_i - D_i) + \frac{1}{2}\sum_{i=1}^N (A_i - D_i)^2.\end{aligned}$$

Taking conditional expectation: $\mathbb{E}[A_i \mid \mathcal{F}_n] = \lambda_i^{\text{pkt}}\tau_n$ (since arrivals are i.i.d. and independent of the current state). For the quadratic term:

$$\mathbb{E}[(A_i - D_i)^2 \mid \mathcal{F}_n] \le \mathbb{E}[A_i^2 \mid \mathcal{F}_n] + D_i^2 \le \bar{\tau}\,\mathbb{E}[(\ell_i^{\text{in}})^2] + \bar{\tau}^2(\lambda_i^{\text{pkt}})^2 + (L_I^{\text{in}})^2,$$

where we used $\mathbb{E}[A_i^2] = \tau_n \mathrm{Var}(\ell_i^{\text{in}}) + (\lambda_i^{\text{pkt}}\tau_n)^2 \le \bar{\tau}\,\mathbb{E}[(\ell_i^{\text{in}})^2] + \bar{\tau}^2(\lambda_i^{\text{pkt}})^2$ and $D_i^2 \le (L_I^{\text{in}})^2$. Summing over i gives $\tilde{B}_0$. □

Lemma C.3 (Expected Per-Frame Cost). *Let $S_0^{(n)} := \sum_i Q_i^+(T_n)$ and $d_n := \sum_{i\in S_n} \ell_i(L_n)$. Under the post-arrival convention of Section 3.1, the expected total cost incurred during frame n satisfies*

$$\mathbb{E}\left[\sum_{t=T_n}^{T_{n+1}-1}\sum_{i=1}^N Q_i^+(t)\,\middle|\,\mathcal{F}_n\right] = \tau_n S_0^{(n)} - (\tau_n - 1)d_n + \frac{\Lambda\,\tau_n(\tau_n-1)}{2}. \tag{23}$$

Proof. At slot T_n, the post-arrival backlog is $S_0^{(n)}$. At slot $T_n + u$ for $u \ge 1$, the post-arrival backlog is $S_0^{(n)} - d_n + \sum_{v=1}^u \sum_i \ell_i^{\text{in}}(T_n + v)$. Summing over $u = 0, 1, \dots, \tau_n - 1$ and taking conditional expectations:

$$\tau_n S_0^{(n)} - (\tau_n - 1)d_n + \Lambda \sum_{u=1}^{\tau_n - 1} u = \tau_n S_0^{(n)} - (\tau_n - 1)d_n + \frac{\Lambda\,\tau_n(\tau_n-1)}{2}.$$

The pre-arrival frame cost follows by subtracting $\Lambda\,\tau_n$ in expectation. □

Lemma C.4 (Threshold Dominance). *Fix a renewal epoch with post-arrival backlogs $\{Q_i^+\}$ and let S be any feasible batch (at most k stations, one buffered prompt each) whose maximum selected prompt length is L, serving length $\ell_i^S \le L$ at each $i \in S$. Then the threshold batch $S(L)$ of Algorithm 1 satisfies*

$$\sum_{i\in S(L)} \big(Q_i^+ + V(\tau(L)-1)\big)\ell_i(L) \;\ge\; \sum_{i\in S} \big(Q_i^+ + V(\tau(L)-1)\big)\ell_i^S,$$

so the drift-plus-penalty score of S is at most $\Phi_V(L)$. Hence the restriction to threshold-parameterized batches is lossless: at every epoch, $\max_{L\in\mathcal{L}} \Phi_V(L)$ upper-bounds the score of every feasible batch, of every dummy-only declared action, and, by linearity of expectation, of every stationary randomized policy's action over the declared-action class of Sections 3–4.

Proof. Both batches share the frame duration $\tau(L) = L + f(k,L)$: by the declared-threshold transition law of Section 3.1, both are evaluated at the common declared threshold L, each incurring frame duration $\tau(L)$ regardless of its realized composition (a batch not using thresholds implicitly declares its realized maximum L). Thus, the penalty terms coincide, and it suffices to compare the weight sums. For each $i \in S$, the definition $\ell_i(L) = \max\{\ell \le L : \text{a prompt of length } \ell \text{ at } i\}$ gives $\ell_i^S \le \ell_i(L)$, yielding termwise domination of the summands of S by the corresponding summands with $\ell_i(L)$. Since $S(L)$ collects the top-k stations by the weight $(Q_i^+ + V(\tau(L)-1))\ell_i(L)$, its weight sum dominates that of every k-subset, in particular that of S. A dummy-only declared action at threshold L has weight sum zero, which the nonnegative weight sum of $S(L)$ dominates, bounding its score by $\Phi_V(L)$ as well. For a randomized policy, applying the inequality per realization and taking expectations completes the proof. □

Remark. The proof of Lemma C.4 applies verbatim when L is any *declared* threshold at least the realized maximum length of S: eligibility $\ell_i^S \le \ell_i(L)$ and the common frame duration $\tau(L)$ are all that is used. We invoke it below in this declared form, consistent with the declared-threshold transition convention of Section 3.1 under which every action consists of a batch together with a declared threshold no smaller than its realized maximum, and the frame is priced at the declared threshold.

Before the main argument we make precise the comparator against which RDPP is measured. In a stationary stable system, long-run *actual* departures equal arrivals, so no policy delivers actual service with persistent slack; the comparator must therefore be defined through *offered* service, which the padded provisioned batch of Section 2.1 makes well defined at every state.

Definition C.5 (Offered-service comparator). An *offered-service comparator* is specified by *score weights* β, a probability distribution over pairs (L,s) with $L \in \mathcal{L}$ and $s \subseteq [N]$, $|s| \le k$; the associated physical comparator draws its actions from the tilted law β° defined below, independently of the past and of the system state. A weighted action declares threshold L (the frame lasts $\tau(L) = L + f(k,L)$ slots by the provisioned transition law of Section 3.1), and offers one transmission opportunity to every station $i \in s$. A station uses its opportunity to transmit its longest buffered prompt of length at most L, removing $\ell_i(L)$ packets; if it holds no eligible prompt ($\ell_i(L) = 0$), the opportunity is a padded slot of the provisioned batch and removes nothing. For a buffer composition x_i of station i, the offered per-slot service rate is

$$r_i^\beta(x_i) \;:=\; \mathbb{E}_{(L,s)\sim\beta}\left[\frac{\mathbf{1}\{i\in s\}\,\ell_i(L;x_i)}{\tau(L)}\right],$$

where $\ell_i(L;x_i)$ makes the dependence on the composition explicit.

Two distributions play distinct roles here. The *score weights* β define the per-frame-ratio (score-form) rate r_i^β above, the quantity the drift analysis consumes. The associated *physical* comparator draws its actions from the tilted law $\beta^\circ(a) \propto \beta(a)/\tau(L)$; by renewal reward, at a frozen composition x_i its long-run offered packet rate to station i is $\mathbb{E}_{\beta^\circ}[\mathbf{1}\{i\in s\}\,\ell_i(L;x_i)]/\mathbb{E}_{\beta^\circ}[\tau(L)] = r_i^\beta(x_i)$, an exact length-bias identity, so slack in the score form and offered slack for the physical comparator are the same condition.

Assumption 2 (Uniform Offered-Service Slack). *There exist $\varepsilon > 0$ and an offered-service comparator β such that, for every station i and every nonempty buffer composition x_i,*

$$r_i^\beta(x_i) \;\ge\; \lambda_i^{pkt} + \varepsilon.$$

This assumption formalizes membership in the offered-slack region C_{off} of Definition 2.1, with service accounted in offered form. We also assume $\alpha_0 > 0$, i.e., in each slot a station generates no prompt with positive probability.

Remark 1 (Achievability via type separation). *Fix weights $x_1,\dots,x_I \ge 0$ with $\sum_j x_j \le 1$ and consider the type-separated stationary policy that spends a fraction x_j of the* time *on class-j frames: a class-j frame declares threshold L_j^{in}, lasts $\tau(L_j^{in})$ slots, and offers one class-j transmission opportunity to each of k stations chosen cyclically over $[N]$, dummy prompts padding stations without one, so it offers kL_j^{in} packets per $\tau(L_j^{in})$ slots. Such policies give a constructive sufficient condition for* ordinary stabilizability *under per-class load inequalities, as instantiated by the stabilizing policies in Appendix A. They do not by themselves establish the composition-uniform condition of Assumption 2; membership in C_{off} requires verifying that condition directly.*

We now prove the main stability result.

Proof of Lemma 4.1. The proof uses a renewal-frame Lyapunov drift argument and proceeds in five steps.

Step 1: Exact score algebra. For an admissible action $a = (L, S)$ at a renewal epoch, write $\tau = \tau(L)$, $W(a) := \sum_{i\in S} Q_i^+(T_n)\,\ell_i(L)$, $d(a) := \sum_{i\in S} \ell_i(L)$, and define the frame-inefficiency rate

$$\psi(a) \;:=\; \frac{\Lambda(\tau-1)}{2} \;-\; \frac{(\tau-1)\,d(a)}{\tau}.$$

Two exact identities follow by direct rearrangement: the score (8) satisfies

$$\Phi_V(a) \;=\; \frac{W(a)}{\tau} \;-\; V\,\psi(a), \tag{24}$$

and Lemma C.3 reads $\mathbb{E}[\,\text{frame-cost}_n \mid \mathcal{F}_n] = \tau\,(S_0^{(n)} + \psi(a))$. Consequently, since the quantities $\sum_i \lambda_i^{\text{pkt}} Q_i^+(T_n)$ and $VS_0^{(n)}$ do not depend on the action, maximizing Φ_V is the same as minimizing

$$R(a) \;:=\; \Big[\sum_i \lambda_i^{\text{pkt}} Q_i^+(T_n) - \frac{W(a)}{\tau}\Big] \;+\; V\big(S_0^{(n)} + \psi(a)\big),$$

and Lemmas C.2 and C.3 combine into

$$\mathbb{E}[\Delta_n + V\,\text{frame-cost}_n \mid \mathcal{F}_n] \;\le\; \tilde{B}_0 + \tau(a)\,R(a). \tag{25}$$

Because RDPP maximizes Φ_V over all thresholds with the top-k batch rule, and every feasible batch is dominated by a threshold batch (Lemma C.4 with its declared-threshold remark), RDPP's action a^{R} minimizes $R(\cdot)$ over all admissible batch actions at the epoch. Partially padded batches are covered as well: all weights $(Q_i^+ + V(\tau-1))\ell_i(L)$ are nonnegative, so the top-k threshold batch dominates every station subset at the same declared threshold, including subsets in which some stations hold no eligible prompt and contribute zero. At epochs where the system is empty, the slot is a unit idle frame and all action-dependent terms vanish.

Step 2: Comparison with the offered-service comparator. Fix (L, s) in the support of the comparator β of Assumption 2. Applying Lemma C.4 to the feasible batch formed by the stations of s holding an eligible prompt, with declared threshold L,

$$\Phi_V(a^{\text{R}}) \;\ge\; \Phi_V(L) \;\ge\; \frac{\sum_{i\in s}\big(Q_i^+(T_n) + V(\tau(L)-1)\big)\ell_i(L)}{\tau(L)} \;-\; \frac{V\Lambda}{2}(\tau(L)-1).$$

By the identity (24), and since every action satisfies $0 \le (\tau-1)d(a)/\tau \le (\bar{\tau}-1)kL_I^{\text{in}}/\underline{\tau}$ and $0 \le \Lambda(\tau-1)/2 \le \Lambda(\bar{\tau}-1)/2$, transferring the V-weighted terms to the right-hand side costs at most a constant multiple of V:

$$\frac{W(a^{\text{R}})}{\tau(a^{\text{R}})} \;\ge\; \frac{\sum_{i\in s} Q_i^+(T_n)\,\ell_i(L)}{\tau(L)} \;-\; V\Gamma_1, \qquad \Gamma_1 := (\bar{\tau}-1)\Big(\frac{kL_I^{\text{in}}}{\underline{\tau}} + \frac{\Lambda}{2}\Big), \qquad \underline{\tau} := \min_{L\in\mathcal{L}} \tau(L) \ge 2.$$

Averaging over $(L, s) \sim \beta$ and applying Assumption 2 station by station (stations with empty buffers contribute zero to both sides),

$$\frac{W(a^{\text{R}})}{\tau(a^{\text{R}})} \;\ge\; \sum_i Q_i^+(T_n)\,r_i^{\beta}(x_i) \;-\; V\Gamma_1 \;\ge\; \sum_i \big(\lambda_i^{\text{pkt}} + \varepsilon\big)\,Q_i^+(T_n) \;-\; V\Gamma_1. \tag{26}$$

The guarantee behind (26) concerns offered service only; no claim is made about the comparator's actual long-run departures, and the padded provisioned batch ensures the offered quantities are well defined at every state.

Step 3: Negative drift with a V-dependent constant. From Lemma C.2 and (26), writing $x := \sum_i \lambda_i^{\text{pkt}} Q_i^+(T_n) - W(a^{\text{R}})/\tau(a^{\text{R}})$ and using $\underline{\tau}x \ge \tau(a^{\text{R}})x$ when $x \le 0$ and $\tau(a^{\text{R}})x \le \bar{\tau}x$ when $x \ge 0$,

$$\mathbb{E}[\Delta_n \mid \mathcal{F}_n] \;\le\; \tilde{B}_0 + \tau(a^{\text{R}})\,x \;\le\; \Gamma(V) \;-\; \varepsilon_0 \sum_i Q_i^+(T_n), \qquad \Gamma(V) := \tilde{B}_0 + \bar{\tau}\,\Gamma_1 V, \quad \varepsilon_0 := \varepsilon\,\underline{\tau}.$$

We record explicitly that the drift constant grows linearly in V: the score comparison transfers the V-weighted penalty terms, which do not cancel between RDPP and the comparator and must be retained. For every fixed V the constant is finite, which is all the Foster criterion requires; the dependence $\Gamma(V) = \Theta(V)$ propagates to the backlog bound in Step 5.

Step 4: Drift telescoping. Let X_n denote the post-arrival state (buffer composition) at the n-th decision epoch; slots at which the system is empty are unit idle frames and are included as epochs, so $\{X_n\}_{n\ge 0}$ is a time-homogeneous Markov chain on a countable state space (RDPP's action is a deterministic function of the state). The drift bound of Step 3 holds at every state, so no irreducibility or reachability property is required; in particular the argument does not need every nonempty frame to serve a packet, and thresholds whose batch is empty (dummy-only provisioned frames, Section 4.1) are covered. Telescoping the drift bound of Step 3 over the first n epochs and dropping the nonnegative terminal Lyapunov term gives, for every n,

$$\frac{1}{n}\sum_{m=0}^{n-1} \mathbb{E}\Big[\sum_i Q_i^+(T_m)\Big] \;\le\; \frac{\Gamma(V)}{\varepsilon_0} + \frac{\mathcal{L}(\boldsymbol{Q}^+(0))}{\varepsilon_0\,n}.$$

Step 5: From epochs to slots. Within frame n, the backlog at any slot is at most $\sum_i Q_i^+(T_n) + \bar{\tau} N L_I^{\text{in}}$, every frame lasts between 1 and $\bar{\tau}$ slots, and at most one frame started before horizon T ends after it; the final, possibly partial, frame is bounded by the same per-frame envelope. Mapping each slot to its frame and using the epoch Cesàro bound of Step 4,

$$\limsup_{T\to\infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}\Big[\sum_i Q_i(t) \Big] \;\le\; \bar{\tau}\, \frac{\Gamma(V)}{\varepsilon_0} \;+\; \bar{\tau} N L_I^{\text{in}} \;=\; O(V) \;<\; \infty.$$

Every step above is a finite-horizon estimate; no renewal-reward theorem is invoked. This proves $\bar{Q}^{\text{RDPP}(V)} < \infty$ for every $V > 0$. □

D Proof of Theorem 4.2

The previous version of this appendix compared RDPP with a stationary policy $\pi_\varepsilon^\star$ of average cost $c^\star + \varepsilon$ through the per-frame conditional inequality $\mathbb{E}[C_n^{\pi_\varepsilon^\star} \mid \mathcal{F}_n] \le (c^\star + \varepsilon)\, \mathbb{E}[\tau_n \mid \mathcal{F}_n]$ at RDPP's visited states. That inequality is invalid: by Lemma C.3 the conditional frame cost contains the standing backlog $S_0^{(n)}$ of the visited state, so a policy's conditional per-frame cost is not controlled by its global average. The corrected analysis proves Theorem 4.2 directly from the exact drift-plus-penalty algebra of Appendix C, and complements it with an exact representation of the total-cost optimality gap through the average-cost optimality equation (ACOE) of Section 3.2.

Proof of Theorem 4.2. Stability and the $O(V)$ backlog bound are Steps 4–5 of the proof of Lemma 4.1. For the efficiency bound, fix any offered-service comparator β (Definition C.5) and let

$$\mathcal{J}_\beta \;:=\; \sup_x \; \mathbb{E}_{(L,s)\sim\beta}\big[\psi(L, s; x)\big]$$

denote its worst-composition expected frame-inefficiency, where the supremum is over buffer compositions and $\psi(L, s; x)$ is the inefficiency rate of the comparator's offered action at composition x; the benchmark of the theorem statement is $\psi^\star = \inf_\beta \mathcal{J}_\beta$. Define RDPP's long-run average frame-inefficiency as

$$\bar{\psi}^{\text{RDPP}(V)} := \limsup_{n_0\to\infty} \frac{\sum_{n<n_0} \mathbb{E}[\tau_n\, \psi(a_n^{\text{R}})]}{\sum_{n<n_0} \mathbb{E}[\tau_n]},$$

with n_0 a deterministic frame count (idle unit frames included); since $1 \le \tau_n \le \bar{\tau}$, frame-count and slot-horizon averages coincide in the limit, and no optional-stopping argument is needed.

Start from (25) with $R(a^{\text{R}}) \le \mathbb{E}_\beta[R(\cdot)]$ (Step 1 of Lemma 4.1's proof), expand $\mathbb{E}_\beta[R]$ using Assumption 2 and $\mathbb{E}_\beta[\psi] \le \mathcal{J}_\beta$, and subtract the exact identity $V\, \mathbb{E}[\text{frame-cost}_n \mid \mathcal{F}_n] = V \tau_n (S_0^{(n)} + \psi(a_n^{\text{R}}))$ from both sides; the $V S_0^{(n)}$ terms cancel exactly and

$$\mathbb{E}[\Delta_n \mid \mathcal{F}_n] + V \tau_n \big(\psi(a_n^{\text{R}}) - \mathcal{J}_\beta\big) \;\le\; \tilde{B}_0 \;-\; \varepsilon_0 \sum_i Q_i^+(T_n) \;\le\; \tilde{B}_0.$$

At empty epochs the relation holds trivially ($\psi = 0$, $\sum_i Q_i^+ = 0$, $\mathcal{J}_\beta \ge 0$). Summing over $n < n_0$, taking expectations, dropping the nonnegative terminal term $\mathbb{E}[\mathcal{L}(\boldsymbol{Q}^+(T_{n_0}))]$ and the initial term $\mathcal{L}(\boldsymbol{Q}^+(0))$, a constant vanishing after division by the horizon, dividing by $V \sum_{n<n_0} \mathbb{E}[\tau_n] \ge V n_0$, and letting $n_0 \to \infty$ gives $\bar{\psi}^{\text{RDPP}(V)} \le \mathcal{J}_\beta + \tilde{B}_0/V$. Taking the infimum over comparators β completes the proof. □

The remainder of this appendix establishes the exact total-cost gap representation announced in Section 4.2. Notation: for a policy π, $c_+^\pi := c^\pi + \Lambda$ is its post-arrival average cost (Section 3.1); the ACOE of Section 3.2 is stated on the post-arrival chain, so its optimal value $c_+^\star$ is a post-arrival cost. The shift is policy-independent, so $c_+^\pi - c_+^\star = c^\pi - c^\star$ and optimality gaps coincide.

Assumption 3 (ACOE regularity). *The ACOE of Section 3.2 (on the post-arrival pair (S^+, c^+)) admits a solution $(c_+^\star, h)$ with h of at most quadratic growth: $|h(S)| \le c_h\big(1 + (\sum_i Q_i^+)^2\big)$ for a constant $c_h < \infty$. In addition, the embedded chain of RDPP is positive recurrent on the closed class it reaches, with invariant distribution μ.*

Discussion. Existence of $(c_+^\star, h)$ is already assumed in Section 3.2 following [2]. The quadratic growth reflects that $h(S)$ measures the transient excess cost of starting at backlog $\boldsymbol{Q}$ rather than in steady state: draining $\|\boldsymbol{Q}\|_1$ packets at the slack rate of Assumption 2 takes $O(\|\boldsymbol{Q}\|_1)$ slots during which the per-slot cost is $O(\|\boldsymbol{Q}\|_1)$, giving an upper quadratic; positivity of costs gives the lower bound. We state it as an assumption rather than derive it.

Lemma D.1 (Stationary second moment). *Under Assumptions 2 and 3, for every $V > 0$ the invariant distribution μ of the embedded chain $\{X_n\}$ satisfies $\mathbb{E}_\mu\big[(\sum_i Q_i^+)^2\big] < \infty$.*

Proof. Per-frame increments are bounded: $|\sum_i Q_i^+(T_{n+1}) - \sum_i Q_i^+(T_n)| \le \bar{\Delta} := \bar{\tau} N L_I^{\text{in}} + k L_I^{\text{in}}$, and $|\Delta_n| \le \bar{\Delta} \sum_i Q_i^+(T_n) + N\bar{\Delta}^2$. Apply the drift bound of Step 3 of Lemma 4.1 to $W := \mathcal{L}^{3/2}$: a second-order Taylor bound gives

$$\mathbb{E}[\Delta W \mid \mathcal{F}_n] \;\le\; \tfrac{3}{2}\, \mathcal{L}^{1/2}\, \mathbb{E}[\Delta_n \mid \mathcal{F}_n] + c_1\big(1 + \mathcal{L}^{1/2}\big) \;\le\; \tfrac{3}{2}\, \mathcal{L}^{1/2}\big(\Gamma(V) - \varepsilon_0 \textstyle\sum_i Q_i^+\big) + c_1\big(1 + \mathcal{L}^{1/2}\big),$$

and since $\mathcal{L}^{1/2} \le \sum_i Q_i^+ \le \sqrt{2N}\, \mathcal{L}^{1/2}$, the right side is at most $-c_2\, \mathcal{L} + c_3$ outside a finite set, for constants $c_2 > 0$, $c_3 < \infty$ depending on V. The Foster criterion with unbounded drift function (e.g., [22], Theorem 14.0.1) yields $\mathbb{E}_\mu[\mathcal{L}] < \infty$, hence the claim. □

Lemma D.2 (Frame ACOE). *Let S be a decision-epoch state and a any admissible action there with frame duration $\tau(a)$. Then*

$$h(S) \;\le\; \mathbb{E}\big[C(S,a) \mid S,a\big] \;-\; c_+^\star\,\tau(a) \;+\; \mathbb{E}\big[h(S') \mid S,a\big],$$

where $C(S,a)$ is the total cost of the frame and S' the state at the next decision epoch. Equality holds when a attains the minimum in the ACOE at S.

Proof. Iterate the per-slot ACOE across the frame. At the decision slot the ACOE gives $h(S) + c_+^\star \le c^+(S) + \mathbb{E}[h(S_1) \mid S,a]$ for the chosen a. At each subsequent slot of the frame the admissible action set is the singleton {idle}, so the minimum is attained trivially and the relation holds with equality. Summing the $\tau(a)$ relations telescopes h along the frame; all conditional expectations are finite because h has quadratic growth (Assumption 3) and per-slot state increments are bounded. □

Proposition D.3 (Exact optimality-gap representation). *Under Assumptions 2 and 3, for every $V > 0$,*

$$c_+^{\mathrm{RDPP}(V)} \;=\; \frac{\mathbb{E}_\mu[C(S,a^{\mathrm{R}}(S))]}{\mathbb{E}_\mu[\tau(a^{\mathrm{R}}(S))]}, \qquad c_+^{\mathrm{RDPP}(V)} - c_+^\star \;=\; c^{\mathrm{RDPP}(V)} - c^\star \;=\; \frac{\mathbb{E}_\mu\big[\mathcal{D}_V(S)\big]}{\mathbb{E}_\mu[\tau(a^{\mathrm{R}}(S))]} \;\ge\; 0,$$

where $a^{\mathrm{R}}(S)$ is RDPP's action at S, μ is the stationary distribution of the embedded chain, and

$$\mathcal{D}_V(S) \;:=\; \mathbb{E}\big[C(S,a^{\mathrm{R}}) \mid S\big] - c_+^\star\tau(a^{\mathrm{R}}) + \mathbb{E}\big[h(S') \mid S,a^{\mathrm{R}}\big] - h(S) \;\ge\; 0$$

is the ACOE disadvantage of RDPP's action.

Proof. Nonnegativity of $\mathcal{D}_V$ is Lemma D.2. By Lemma D.1 and Assumption 3, h is μ-integrable, and $\mathbb{E}[C(S,a^{\mathrm{R}}) \mid S] \le \bar\tau \sum_i Q_i^+ + \bar\tau^2 N L_I^{\mathrm{in}}$ is μ-integrable as well. Taking $\mathbb{E}_\mu$ of the definition of $\mathcal{D}_V$ and using invariance, $\mathbb{E}_\mu[\mathbb{E}[h(S') \mid S,a^{\mathrm{R}}]] = \mathbb{E}_\mu[h]$, so $\mathbb{E}_\mu[\mathcal{D}_V] = \mathbb{E}_\mu[C] - c_+^\star\,\mathbb{E}_\mu[\tau]$. For the first display, the chain is positive recurrent with μ-integrable frame cost and bounded frame length, so the ratio-ergodic theorem for Markov chains (e.g., [22]) gives $\lim_{T\to\infty} \frac{1}{T}\sum_{t<T}\mathbb{E}[\sum_i Q_i^+(t)] = \mathbb{E}_\mu[C]/\mathbb{E}_\mu[\tau]$; the final partial frame at any horizon is bounded by the μ-integrable envelope $\bar\tau\sum_i Q_i^+(T_{N(T)}) + \bar\tau^2 N L_I^{\mathrm{in}}$ and vanishes after division by T. Combining the two displays completes the proof. □

Remark (why the theorem is stated for frame efficiency). Proposition D.3 shows that a total-cost bound of the form $c_+^{\mathrm{RDPP}(V)} \le c_+^\star + O(1/V)$ (equivalently, the same bound for the pre-arrival costs) would require $\mathbb{E}_\mu[\mathcal{D}_V] = O(1/V)$, i.e., RDPP's action to be, on average under its own stationary distribution, within $O(1/V)$ of the ACOE minimizer. The drift-plus-penalty machinery does not deliver this bound: as $V \to \infty$, RDPP's action at any fixed state converges to the queue-blind rule maximizing $(\tau(L)-1)\,d/\tau(L) - \Lambda(\tau(L)-1)/2$, the myopic one-frame cost minimizer, whose ACOE disadvantage is in general bounded away from zero. The frame-efficiency benchmark of Theorem 4.2 is the component of the cost that the score provably optimizes, and the representation above isolates exactly what a stronger total-cost claim would additionally require.

E Proof of Theorem 5.1

Throughout, T_n denotes the n-th renewal epoch, and $\mathcal{F}_n$ the σ-algebra generated by the history up to and including the decision (L_n, S_n) taken at T_n (outputs of the batch are not $\mathcal{F}_n$-measurable). For an action $a = (L,S)$ let $\tilde\ell(a) := \max_{i\in S}\ell_i(L)$ denote the maximum true selected input length; by the eligibility rule $Q_\delta(\ell) \le L$ of Algorithm 2 we have $\tilde\ell(a) < L + \delta$. The realized frame duration is driven by the realized batch,

$$\tau_n \;=\; \tilde\ell(a_n) + f\big(k,\tilde\ell(a_n)\big) + D\big(k,\tilde\ell(a_n),\ell^{\mathrm{out}}_{\max,n}\big), \qquad \ell^{\mathrm{out}}_{\max,n} := \max_{i\in S_n}\ell_i^{\mathrm{out}},$$

and we write $\bar m(a) := \mathbb{E}[\tau_n \mid \mathcal{F}_n]$ for its conditional mean. We call the action $a = (L,S)$ *saturated* if its computational input maximum reaches the declared type: either $\tilde\ell(a) \ge L$, or the sequence-length padding of Appendix F raises one selected prompt's input tensor to length L for computation, in which case τ_n is given by the display above with $\tilde\ell(a_n)$ replaced by L while queue service still removes only the true packets. Any feasible batch S is saturated of the first kind at the threshold $L = Q_\delta(\tilde\ell(S))$, a relabeling that changes neither the served prompts nor the realized frame.

Lemma E.1 (Estimation bias of the reference profile). *Let Assumption 1 hold and set $\beta := \Delta_g + \kappa\delta$. For any action $a = (L,S)$ with $S \ne \emptyset$: (a) $\bar m(a) \le \hat\tau(L) + \beta$; (b) if a is saturated, also $\bar m(a) \ge \hat\tau(L) - \beta$; (c) $\mathrm{Var}(\tau_n \mid \mathcal{F}_n) \le \Delta_g^2/4$. For the dummy-only action at type L ($S = \emptyset$), the frame duration is drawn from the reference profile itself, so (a)–(c) hold with zero bias and the action is saturated by convention.*

Proof. (a) Since $\tilde\ell < L + \delta$ and $f(k,\cdot)$ is nondecreasing and L_f-Lipschitz, $\tilde\ell + f(k,\tilde\ell) \le L + f(k,L) + (1+L_f)\delta$. For the decode term, monotonicity of D in its second argument and the L_D-Lipschitz property give, for every realization o of $\ell^{\mathrm{out}}_{\max,n}$, $D(k,\tilde\ell,o) \le D(k,L,o) + L_D\delta$. Moreover, for fixed (k,L) the map $o \mapsto D(k,L,o)$ takes values in an interval of width at most Δ_g, and $\hat D(L)$ lies in the same interval; hence $\mathbb{E}[D(k,L,\ell^{\mathrm{out}}_{\max,n}) \mid \mathcal{F}_n] \le \hat D(L) + \Delta_g$, regardless of the realized batch's input profile or fill. Summing the three bounds yields (a).

(b) For saturated a, $\tilde\ell \ge L$ gives $\tilde\ell + f(k,\tilde\ell) \ge L + f(k,L)$ and $D(k,\tilde\ell,o) \ge D(k,L,o)$ by monotonicity, and the same interval argument gives the lower bias $-\Delta_g$. For a padded action (Appendix F) the computational input maximum equals L exactly, and (a) and (b) hold with $\tilde\ell$ replaced by L.

(c) Conditional on $\mathcal{F}_n$, all randomness in τ_n enters through $D(k,\tilde\ell,\ell^{\mathrm{out}}_{\max,n})$, which ranges over an interval of width at most Δ_g; Popoviciu's inequality bounds the variance by $\Delta_g^2/4$. □

Lemma E.2 (Score distortion under the robustness weighting). *For an action $a = (L, S)$ define $W(a) := \sum_{i\in S}\big(Q_i^+ + V(\hat\tau(L)-1)\big)\ell_i(L) \ge 0$, $\mathrm{pen}(L) := \frac{V\Lambda}{2}(\hat\tau(L)-1) \ge 0$, and the scores*

$$G(a) := \frac{W(a)}{\hat\tau(L)} - \mathrm{pen}(L), \qquad G^\eta(a) := \frac{W(a)}{\hat\tau(L) + \eta\hat\sigma_\tau(L)} - \mathrm{pen}(L),$$

so that $\Phi_V^\eta(L) = G^\eta(L, S(L))$ in (9). Then, with $\theta_\eta := \tau_{\min}/(\tau_{\min} + \eta\Delta_g) \in (0, 1]$,

$$G(a) \;\ge\; G^\eta(a) \;\ge\; \theta_\eta\, G(a) - (1-\theta_\eta)\,\mathrm{pen}(L) \quad \textit{for every action } a.$$

Moreover, since $S(L)$ maximizes $W(L, S)$ over feasible batches at threshold L and the denominator of G^η does not depend on S, the batch-selection and threshold-selection steps of Algorithm 2 jointly maximize G^η over all actions: $G^\eta(a_n) \ge G^\eta(a)$ for every feasible a with $L \in \mathcal{G}_\delta$.

Proof. The first inequality holds since $W \ge 0$ and $\hat\tau + \eta\hat\sigma_\tau \ge \hat\tau$. For the second, write $G^\eta(a) + \mathrm{pen}(L) = \frac{\hat\tau(L)}{\hat\tau(L)+\eta\hat\sigma_\tau(L)} \cdot \frac{W(a)}{\hat\tau(L)} \ge \theta_\eta\big(G(a) + \mathrm{pen}(L)\big)$, using $\hat\sigma_\tau \le \Delta_g$, $\hat\tau \ge \tau_{\min}$, and $W/\hat\tau = G + \mathrm{pen} \ge 0$. The maximization claim is immediate from the listing of Algorithm 2. □

Proof of Theorem 5.1. **Step 1: Per-frame Lyapunov drift.** Lemma C.1 holds unchanged with the random frame length τ_n. As in Lemma C.2, using that arrivals are i.i.d. and independent of (a_n, τ_n), and that $\tau_n \le \bar\tau$,

$$\mathbb{E}[\Delta_n \mid \mathcal{F}_n] \le \tilde B_0' + \sum_{i=1}^N Q_i^+(T_n)\Big(\lambda_i^{\mathrm{pkt}}\,\bar m(a_n) - \mathbf{1}\{i \in S_n\}\,\ell_i(L_n)\Big), \tag{27}$$

with $\tilde B_0' := \frac{1}{2}\sum_i\big[\bar\tau\,\mathbb{E}[(\ell_i^{\mathrm{in}})^2] + \bar\tau^2(\lambda_i^{\mathrm{pkt}})^2 + (\ell_{\max}^{\mathrm{in}})^2\big] < \infty$. By Lemma E.1(a), $\bar m(a_n) \le \hat\tau(L_n) + \beta$. Dividing by $\hat\tau(L_n) \ge \tau_{\min}$ and using $\sum_{i\in S_n}\ell_i(L_n) \le k\ell_{\max}^{\mathrm{in}}$,

$$\frac{\mathbb{E}[\Delta_n \mid \mathcal{F}_n]}{\hat\tau(L_n)} \le C_1 + \Big(1 + \frac{\beta}{\tau_{\min}}\Big)\sum_i Q_i^+\lambda_i^{\mathrm{pkt}} - G(a_n), \qquad C_1 := \frac{\tilde B_0'}{\tau_{\min}} + Vk\ell_{\max}^{\mathrm{in}}, \tag{28}$$

where we used $\sum_{i\in S_n} Q_i^+\ell_i(L_n) = W(a_n) - V(\hat\tau(L_n)-1)\sum_{i\in S_n}\ell_i(L_n)$ and $W(a_n)/\hat\tau(L_n) = G(a_n) + \mathrm{pen}(L_n) \ge G(a_n)$.

Step 2: Comparator and its score value. By the score-form offered-slack hypothesis of Theorem 5.1, there are score weights p over grid actions (L, s), $L \in \mathcal{G}_\delta$, in the *score-form offered-service* sense of Definition C.5: a weighted action declares threshold L; stations of s without an eligible real prompt contribute no service, one selected real prompt is sequence-padded to L for computation when the real batch is nonempty with computational maximum below L, and an empty real batch processes a dummy request at L, so every weighted action is saturated and queue service removes only true packets.

The weights are a bookkeeping object of the drift analysis; the associated physical comparator at composition x draws actions from the tilted law $p_x^\circ(a) \propto p(a)/\bar m(a; x)$, a stationary *state-dependent* randomized policy whose renewal-reward offered rate at frozen composition x equals the score-form rate below. In the robust setting $\bar m(a; x)$ varies with the composition through the selected requests' input-conditioned output laws, so the tilting is composition-dependent and no state-blind physical comparator is claimed.

Matching the composition-uniform slack form of Assumption 2, the score-form offered rate satisfies, for every station i with nonempty buffer and every buffer composition x,

$$r_i^{\pi_\varepsilon}(x) := \sum_a p(a)\,\frac{\mathbb{E}[\mathbf{1}\{i \in s\}\,\ell_i(L; x_i) \mid a]}{\bar m(a; x)} \;\ge\; \lambda_i^{\mathrm{pkt}} + \varepsilon,$$

where $\bar m(a; x)$ is the conditional mean frame duration of the offered action at composition x; this is the quantitative content of the slack hypothesis in the theorem statement, and the offered accounting makes no claim about the comparator's actual long-run departures.

By Lemma E.1(b), $\hat\tau(L_a) \le \bar m(a) + \beta$ for saturated actions, hence $1/\hat\tau(L_a) \ge (1 - \beta/\tau_{\min})/\bar m(a)$, and therefore, averaging over p,

$$\sum_a p(a)\,G(a) \;\ge\; \Big(1 - \frac{\beta}{\tau_{\min}}\Big)\sum_i Q_i^+\big(\lambda_i^{\mathrm{pkt}} + \varepsilon\big) \;-\; \overline{\mathrm{pen}}, \qquad \overline{\mathrm{pen}} := \frac{V\Lambda}{2}\big(\max_L \hat\tau(L) - 1\big),$$

where we dropped the nonnegative V-part of W in the numerator.

Step 3: From score maximization to drift, with distortion. By Lemma E.2 and max ≥ average,

$$G(a_n) \ge G^\eta(a_n) \ge \sum_a p(a)\,G^\eta(a) \ge \theta_\eta\sum_a p(a)\,G(a) - (1-\theta_\eta)\,\overline{\mathrm{pen}} \ge \theta_\eta\Big(1 - \frac{\beta}{\tau_{\min}}\Big)\sum_i Q_i^+(\lambda_i^{\mathrm{pkt}} + \varepsilon) - \overline{\mathrm{pen}}.$$

Substituting into (28), the coefficient of Q_i^+ is $\lambda_i^{\mathrm{pkt}}(1+\beta/\tau_{\min}) - \theta_\eta(1-\beta/\tau_{\min})(\lambda_i^{\mathrm{pkt}} + \varepsilon)$. Using $\theta_\eta \ge 1 - \eta\Delta_g/\tau_{\min}$ and $(1-u)(1-b) \ge 1-u-b$ with $u := \eta\Delta_g/\tau_{\min}$, $b := \beta/\tau_{\min}$, this coefficient is at most $(u + 2b)(\lambda_i^{\mathrm{pkt}} + \varepsilon) - \varepsilon$, and the slack condition of the theorem statement, $\eta\Delta_g + 2\beta \le \varepsilon\tau_{\min}/(2(\lambda_{\max}^{\mathrm{pkt}} + \varepsilon))$, makes it at most $-\varepsilon/2$. Hence

$$\mathbb{E}[\Delta_n \mid \mathcal{F}_n] \;\le\; \Gamma \;-\; \varepsilon_1\sum_i Q_i^+(T_n), \qquad \Gamma := \Big(\max_L \hat\tau(L)\Big)\big(C_1 + 2\,\overline{\mathrm{pen}}\big), \quad \varepsilon_1 := \frac{\varepsilon\,\tau_{\min}}{2}.$$

Step 4: Grid-sampled service bound. Any stationary grid policy serves at most $k\tilde{\ell}$ packets in a frame of duration at least $\tilde{\ell} + f(k, \tilde{\ell})$ (as $D \geq 0$), so its service rate is at most $\max_\ell k\ell/(\ell + f(k,\ell)) = \mu^\star$; dropping the decode term thus yields a valid outer bound. Under Assumption 1 the map $\ell \mapsto k\ell/(\ell + f(k,\ell))$ is Lipschitz on the compact interval, and the maximizing $L^\star$ satisfies $|L^\star - Q_\delta(L^\star)| < \delta$, giving $\mu^\star_\delta \geq \mu^\star - O(\delta)$, as stated after Theorem 5.1. Conversely, restricting threshold labels to $\mathcal{G}_\delta$ does not reduce the set of physically executable batches: a batch with maximum true length $\tilde{\ell}$ can be labeled by the grid threshold $Q_\delta(\tilde{\ell})$, since $\ell \leq \tilde{\ell}$ implies $Q_\delta(\ell) \leq Q_\delta(\tilde{\ell})$, without changing its selected prompts or realized frame duration. Thus quantization introduces no additional physical-action restriction; no containment between the grid-restricted offered-slack class and the interior of C is claimed, and the $O(\delta)$ terms in the guarantees arise from score-estimation bias entering through $\kappa\delta$ in Lemma E.1, while the algorithm's slack requirement $O(\delta + (1+\eta)\Delta_g)$ of Step 3 vanishes as $\delta \to 0$ and $\Delta_g \to 0$.

Step 5: Foster–Lyapunov conclusion. The drift condition of Step 3 holds outside the compact set $\{\boldsymbol{Q} : \sum_i Q_i \leq \Gamma/\varepsilon_1\}$, and all frame lengths are bounded by $\bar{\tau}$. No irreducibility structure is needed, for discrete or continuous input distributions alike: since the negative drift term is bounded on the exception set, the drift bound of Step 3 holds in the global form $\mathbb{E}[\Delta\mathcal{L} \mid \mathcal{F}_n] \leq \Gamma' - \varepsilon_1 \sum_i Q_i^+(T_n)$ for a finite constant Γ'; telescoping it over the first n frames and discarding the nonnegative terminal Lyapunov term gives $\limsup_n \frac{1}{n}\sum_{m<n}\mathbb{E}[\sum_i Q_i^+(T_m)] \leq \Gamma'/\varepsilon_1 < \infty$, a finite epoch-average backlog. Within-frame arrivals contribute at most $\bar{\tau}$ slots of bounded-mean increments per frame, so the slot-average backlog is finite as well, which is the stability criterion $\bar{Q} < \infty$ of Section 2.2, obtained without Harris-recurrence assumptions. □

Remark 2 (Why the theorem imposes a robustness cap). *For large η, variance weighting can dominate the service-efficiency component of the score and reverse threshold preferences. When η is large, the denominator $\hat{\tau}(L) + \eta\hat{\sigma}_\tau(L)$ suppresses every threshold with $\hat{\sigma}_\tau(L) > 0$, and the threshold choice is governed by $\hat{\sigma}_\tau$ rather than by frame efficiency: R-RDPP then persistently prefers an inclusive threshold whose batches mix long and short prompts, reproducing the max-based frame inflation that destabilizes max-weight scheduling in Lemma 2.3. A two-length instance with uncertain short-prompt decode ($\hat{\sigma}_\tau(L_{short}) > 0$) and deterministic long-prompt decode ($\hat{\sigma}_\tau(L_{long}) = 0$) shows, in numerical experiments at arrival rates satisfying the slack hypothesis, bounded backlog for small η and linearly diverging backlog once η exceeds a slack-dependent level; the displayed cap is a sufficient, slack-dependent restriction and is not claimed to be tight. The same mechanism, with $\eta\hat{\sigma}_\tau$ replaced by the estimation bias of Lemma E.1, shows that the $O(\Delta_g)$ margin at $\eta = 0$ is also not an artifact of the analysis.*

F Proof of Theorem 5.2

This appendix proves Theorem 5.2 by running the exact drift-plus-penalty telescoping of Appendix D inside the robust setting of Appendix E: the bias lemma (Lemma E.1), the distortion lemma (Lemma E.2), and Popoviciu's variance bound replace the exact score algebra available in the deterministic case. As in Appendix D, the guarantee is stated for frame efficiency; the reasons a total-cost bound of the form $c^\star + O(1/V)$ is structurally unavailable (Proposition D.3 and the remark following it) apply verbatim here.

Notation and scoring convention. We use the notation of Appendix E: $\bar{m}(a)$, $\hat{\tau}(L)$, $\hat{\sigma}_\tau(L)$, saturation, the bias constant $\beta := \Delta_g + \kappa\delta$, and $\theta_\eta = \tau_{\min}/(\tau_{\min} + \eta\Delta_g)$. We abbreviate $b := \beta/\tau_{\min}$, $u := \eta\Delta_g/\tau_{\min}$, $S_0^{(n)} := \sum_i Q_i^+(T_n)$, $\Lambda_Q := \sum_i \lambda_i^{\mathrm{pkt}} Q_i^+(T_n)$, and, for an action $a = (L, S)$,

$$W_Q(a) := \sum_{i\in S} Q_i^+(T_n)\,\ell_i(L) \;\geq 0, \qquad d(a) := \sum_{i\in S}\ell_i(L),$$

so that the weight of Lemma E.2 splits as $W(a) = W_Q(a) + V(\hat{\tau}(L) - 1)\,d(a)$. The slack condition of Theorem 5.1 reads $u + 2b \leq \varepsilon/\big(2(\lambda_{\max}^{\mathrm{pkt}} + \varepsilon)\big) < 1/2$; in particular $\beta \leq \tau_{\min}/4$, so this hypothesis of the theorem statement is automatic. Since $D \geq 0$ and f is nondecreasing, every realized frame lasts at least $\tau_{\min}$ slots, hence $\bar{m}(a) \geq \tau_{\min}$ for every action, and $\hat{\tau}(L) \in [\tau_{\min}, \bar{\tau}]$ for every threshold. We write

$$c_\psi := \frac{\Lambda}{2} + \frac{k\,\ell_{\max}^{\mathrm{in}}}{\tau_{\min}^2}, \qquad \bar{\psi}_{\max} := \frac{\Lambda(\bar{\tau} - 1)}{2} + k\,\ell_{\max}^{\mathrm{in}}.$$

Throughout this appendix R-RDPP is run with *saturated scoring*: for each grid type $L \in \mathcal{G}_\delta$, the top-k batch by weights among stations with eligible prompts is scored at L (an empty selection denotes the dummy-only provisioned frame of type L, its output drawn from the reference profile at L), and the score G^η of Lemma E.2 is maximized over these actions, at unchanged complexity $O(I_\delta N \log k)$. If the longest selected true length reaches L (its grid label is L), the selected action is saturated as is. Otherwise the server pads the *input tensor* of one selected request to sequence length L for computation only: the executed frame physically runs at type L; no provisioned slot is consumed and the batch size is unchanged; queue service removes exactly the true packets of the selected prompts; and each output follows its own output law. Hence every executed action is saturated in the sense of Lemma E.1(b), and the scored class contains every comparator action of Definition F.1. The device is consistent with the declared-threshold convention (remark preceding Definition C.5) and alters physical behavior only when the chosen grid type exceeds every selected true length.

Theorem 5.1 and its proof hold verbatim under this convention, since that proof uses only Lemma E.1(a) for the selected action and compares against saturated comparator actions, which the saturated maximization dominates exactly as before. Remark 3 explains why some such device is needed.

The benchmark. We first fix the comparator class, harmonizing Definition C.5 with the grid and with output uncertainty.

Definition F.1 (Grid offered-service comparator and the benchmark $J^\star_\delta$). A *grid offered-service comparator* is an offered-service comparator in the score-form sense of Definition C.5 with thresholds restricted to $\mathcal{G}_\delta$: score weights p over pairs (L, s) with $L \in \mathcal{G}_\delta$ and $|s| \le k$; a weighted action declares threshold L; stations of s with no eligible real prompt contribute no queue service. If the resulting real batch is nonempty but its computational input maximum is below L, one selected real prompt is sequence-padded to L for computation only; if it is empty, a dummy request of input length L is processed. In either case queue service removes only true packets.

The associated physical comparator at composition x draws actions from the tilted law $p^\circ_x(a) \propto p(a)/\bar m(a;x)$, a stationary state-dependent randomized policy whose renewal-reward offered rate at frozen composition x equals the score-form rate $r^p_i(x)$ below. By this convention, every comparator action's computational input maximum equals its declared L, so it is saturated and Lemma E.1(b) applies to it.

The comparator is ε-*slack* if for every buffer composition x and every station i with nonempty buffer,

$$r^p_i(x) := \sum_{a=(L,s)} p(a)\,\frac{\mathbf{1}\{i \in s\}\,\ell_i(L;x_i)}{\bar m(a;x)} \;\ge\; \lambda^{\mathrm{pkt}}_i + \varepsilon,$$

the grid form of Assumption 2 and the quantitative content of the slack hypothesis of Theorem 5.1, as instantiated in Step 2 of the proof of that theorem. Its worst-composition expected inefficiency is

$$\mathcal{J}_p := \sup_x \sum_a p(a)\,\psi(a;x),$$

where $\psi(a;x)$ is evaluated at the comparator's *own* conditional mean duration $\bar m(a;x)$ and served total $d(a;x)$, and the benchmark is $J^\star_\delta := \inf_p \mathcal{J}_p$ over ε-slack grid comparators. Since at the empty composition every offered slot is padded and $d = 0$, the supremum includes the value $\sum_a p(a)\Lambda(\bar m(a) - 1)/2 \ge 0$, so $\mathcal{J}_p \ge 0$.

Quantization affects the comparator only through the constant $\kappa\delta$ inside β: its actions are saturated grid actions, physically identical, up to input padding, to feasible batches of the continuous system.

Lemma F.2 (Robust frame cost identity). *Let $C_n := \sum_{t=T_n}^{T_{n+1}-1} \sum_i Q^+_i(t)$ be the total cost of frame n, taken under any action a with conditional mean duration $\bar m(a)$ and served total $d(a)$. Then*

$$\mathbb{E}[C_n \mid \mathcal{F}_n] = \bar m(a)\Big(S^{(n)}_0 + \psi(a) + v(a)\Big), \qquad 0 \;\le\; v(a) \;\le\; \frac{\Lambda\,\Delta_g^2}{8\,\tau_{\min}}.$$

Proof. Conditioning on τ_n and applying Lemma C.3 pathwise (arrivals are independent of (a, τ_n)),

$$\mathbb{E}[C_n \mid \mathcal{F}_n] = \bar m\, S^{(n)}_0 - (\bar m - 1)\,d + \frac{\Lambda}{2}\Big(\bar m(\bar m - 1) + \mathrm{Var}(\tau_n \mid \mathcal{F}_n)\Big),$$

using $\mathbb{E}[\tau_n^2 \mid \mathcal{F}_n] = \bar m^2 + \mathrm{Var}(\tau_n \mid \mathcal{F}_n)$. Dividing the first three terms by $\bar m$ reproduces $S^{(n)}_0 + \psi(a)$ exactly, and the variance term is $\bar m\, v(a)$ by definition. The bound on v combines $\mathrm{Var}(\tau_n \mid \mathcal{F}_n) \le \Delta_g^2/4$ (Lemma E.1(c)) with $\bar m(a) \ge \tau_{\min}$. □

The next lemma is the bridge between the estimated score the algorithm maximizes and the true score the cost identity charges. Define the *true per-slot score* of an action, the analogue of the identity (24) at conditional means,

$$\widetilde\Phi(a) := \frac{W_Q(a)}{\bar m(a)} \;-\; V\,\psi(a).$$

Lemma F.3 (Bias conversion of the score). *Let Assumption 1 hold, let $\beta \le \tau_{\min}/4$, and let $a = (L, S)$ be a* saturated *action, so that $|\hat\tau(L) - \bar m(a)| \le \beta$ by Lemma E.1. Then, with G, G^η,* pen *as in Lemma E.2:*

(i) *(comparator side)* $G(a) \;\ge\; (1-b)\,\dfrac{W_Q(a)}{\bar m(a)} \;-\; V\,\psi(a) \;-\; V\beta c_\psi$;

(ii) *(policy side)* $\widetilde\Phi(a) \;\ge\; (1-b)\,G^\eta(a) \;-\; b\,\mathrm{pen}(L) \;-\; V\beta c_\psi$.

Moreover $|\psi(a)| \le \bar\psi_{\max}$ for every action.

Proof. The last claim is immediate from $\bar m \le \bar\tau$, $(\bar m - 1)/\bar m \le 1$, and $d \le k\ell^{\mathrm{in}}_{\max}$.

(i) Write $\hat\tau = \hat\tau(L)$, $\bar m = \bar m(a)$, $d = d(a)$, and compare $G(a) = W_Q/\hat\tau + V(\hat\tau - 1)d/\hat\tau - (V\Lambda/2)(\hat\tau - 1)$ term by term with $(1-b)W_Q/\bar m - V\psi(a) = (1-b)W_Q/\bar m + V(\bar m - 1)d/\bar m - (V\Lambda/2)(\bar m - 1)$. First, $\hat\tau \le \bar m + \beta$ and $\bar m + \beta \ge \hat\tau \ge \tau_{\min}$ give

$$\frac{1}{\hat\tau} \;\ge\; \frac{1}{\bar m + \beta} \;\ge\; \frac{1-b}{\bar m}, \qquad \text{hence} \qquad \frac{W_Q}{\hat\tau} \;\ge\; (1-b)\,\frac{W_Q}{\bar m}. \tag{29}$$

Second, $\frac{\hat\tau - 1}{\hat\tau} - \frac{\bar m - 1}{\bar m} = \frac{1}{\bar m} - \frac{1}{\hat\tau} = \frac{\hat\tau - \bar m}{\hat\tau \bar m} \ge -\beta/\tau^2_{\min}$, so the service terms differ by at least $-Vd\beta/\tau^2_{\min} \ge -V\beta k\ell^{\mathrm{in}}_{\max}/\tau^2_{\min}$. Third, $-(V\Lambda/2)(\hat\tau - 1) \ge -(V\Lambda/2)(\bar m - 1) - (V\Lambda/2)\beta$. Summing the three bounds gives (i) with $c_\psi = \Lambda/2 + k\ell^{\mathrm{in}}_{\max}/\tau^2_{\min}$.

(ii) Write $G^\eta(a) = W/(\hat\tau+\eta\hat\sigma_\tau) - \mathrm{pen}(L)$ with $W = W_Q + V(\hat\tau-1)d \ge W_Q$ and compare with $\widetilde\Phi(a) = W_Q/\bar m + V(\bar m-1)d/\bar m - (V\Lambda/2)(\bar m-1)$, in three parts. *Queue part:* $\bar m \le \hat\tau+\beta \le \hat\tau+\eta\hat\sigma_\tau+\beta$ and $\hat\tau+\eta\hat\sigma_\tau+\beta \ge \tau_{\min}$ give, as in (29), $W_Q/\bar m \ge (1-b)\,W_Q/(\hat\tau+\eta\hat\sigma_\tau)$, and since $W_Q \le W$,

$$\frac{W_Q}{\bar m} \;\ge\; \frac{W_Q}{\hat\tau+\eta\hat\sigma_\tau} - b\,\frac{W}{\hat\tau+\eta\hat\sigma_\tau} \;=\; \frac{W_Q}{\hat\tau+\eta\hat\sigma_\tau} - b\big(G^\eta(a)+\mathrm{pen}(L)\big).$$

Service part: using $\frac{\hat\tau-1}{\hat\tau+\eta\hat\sigma_\tau} \le \frac{\hat\tau-1}{\hat\tau} = 1-\frac{1}{\hat\tau}$ and $\frac{\bar m-1}{\bar m} = 1-\frac{1}{\bar m}$,

$$Vd\,\frac{\bar m-1}{\bar m} - Vd\,\frac{\hat\tau-1}{\hat\tau+\eta\hat\sigma_\tau} \;\ge\; Vd\Big(\frac{1}{\hat\tau}-\frac{1}{\bar m}\Big) \;=\; -Vd\,\frac{(\hat\tau-\bar m)^+}{\hat\tau\bar m} \;\ge\; -\frac{V\beta\,k\ell^{\mathrm{in}}_{\max}}{\tau^2_{\min}},$$

where the middle expression is nonnegative when $\bar m \ge \hat\tau$ and saturation caps $(\hat\tau-\bar m)^+ \le \beta$ otherwise. *Penalty part:* $-(V\Lambda/2)(\bar m-1) \ge -(V\Lambda/2)(\hat\tau-1) - (V\Lambda/2)\beta$. Adding the three parts and collecting the two $V\beta$-terms into $V\beta c_\psi$ yields (ii); the V-part of the numerator enters only through the harmless $-b(G^\eta+\mathrm{pen})$ correction. □

Proof of Theorem 5.2. Fix an ε-slack grid comparator p (Definition F.1); at the end we take the infimum over p. All displays below hold at every decision epoch n at which the system is nonempty; at empty epochs the frame is a unit idle slot, $\psi = v = 0$, $S_0^{(n)} = 0$, and every display below holds trivially because $\mathcal{J}_p \ge 0$ and the error constants are nonnegative.

Step 1: Master inequality. *The drift-plus-penalty expression is controlled by the true per-slot score.* Adding V times the identity of Lemma F.2 to the drift bound (27) of Appendix E, and writing a_n for R-RDPP's action, $\bar m_n := \bar m(a_n)$,

$$\mathbb{E}[\Delta_n + VC_n \mid \mathcal{F}_n] \;\le\; \tilde B_0' \;+\; \frac{V\Lambda}{2}\,\mathrm{Var}(\tau_n \mid \mathcal{F}_n) \;+\; \bar m_n\,R(a_n), \qquad R(a) := \Lambda_Q - \widetilde\Phi(a) + VS_0^{(n)}. \tag{30}$$

Indeed, the drift bound contributes $\bar m_n\Lambda_Q - W_Q(a_n)$, and $V\,\mathbb{E}[C_n \mid \mathcal{F}_n] = V\bar m_n(S_0^{(n)} + \psi(a_n)) + \frac{V\Lambda}{2}\mathrm{Var}(\tau_n \mid \mathcal{F}_n)$; grouping the action terms as $-W_Q(a_n)/\bar m_n + V\psi(a_n) = -\widetilde\Phi(a_n)$ gives (30). Note that both the drift bound and the cost identity are stated at the *same* conditional mean $\bar m_n$; the estimate $\hat\tau$ has not entered yet.

Step 2: The comparator's estimated score is large. *Slack pays for the queue part; the comparator's own inefficiency is all it charges.* Every comparator action is saturated, so Lemma F.3(i) applies to it; averaging over p at the current composition x, the ε-slack property gives $\sum_a p(a)W_Q(a)/\bar m(a) = \sum_i Q_i^+\, r_i^p(x) \ge \sum_i Q_i^+(\lambda_i^{\mathrm{pkt}}+\varepsilon) =: \Sigma$, and $\sum_a p(a)\psi(a;x) \le \mathcal{J}_p$, hence

$$\sum_a p(a)\,G(a) \;\ge\; (1-b)\,\Sigma \;-\; V\mathcal{J}_p \;-\; V\beta c_\psi. \tag{31}$$

Step 3: Distortion transfer to the maximizer. *The η-weighting costs a θ_η shrinkage, nothing more.* Saturated scoring maximizes G^η over a class containing every comparator action, so $G^\eta(a_n) \ge \sum_a p(a)G^\eta(a)$. By Lemma E.2, $G^\eta(a) \ge \theta_\eta G(a) - (1-\theta_\eta)\mathrm{pen}(L_a)$ for each a; averaging, using $\mathrm{pen}(L) \le \overline{\mathrm{pen}} := \frac{V\Lambda}{2}(\bar\tau-1)$, $1-\theta_\eta \le u$, $\theta_\eta \ge 1-u$, and, on the possibly negative terms of (31), $|\mathcal{J}_p| \le \bar\psi_{\max}$ and $\theta_\eta \le 1$,

$$G^\eta(a_n) \;\ge\; (1-u)(1-b)\,\Sigma \;-\; V\mathcal{J}_p \;-\; Ve_1, \qquad e_1 := u\Big(\bar\psi_{\max} + \frac{\Lambda(\bar\tau-1)}{2}\Big) + \beta c_\psi. \tag{32}$$

Step 4: Policy-side conversion and the queue-error accounting. *Every queue-proportional error is absorbed by the slack; every V-proportional error is a constant.* The selected action a_n is saturated by the scoring convention, so Lemma F.3(ii) and then (32) give

$$\widetilde\Phi(a_n) \;\ge\; (1-b)\,G^\eta(a_n) - Ve_2 \;\ge\; (1-u)(1-b)^2\,\Sigma - V\mathcal{J}_p - V\big(e_1+e_2+b\bar\psi_{\max}\big), \qquad e_2 := b\,\frac{\Lambda(\bar\tau-1)}{2} + \beta c_\psi,$$

where $(1-b)(-V\mathcal{J}_p) \ge -V\mathcal{J}_p - bV\bar\psi_{\max}$ handles the sign of $\mathcal{J}_p$. Using $(1-u)(1-b)^2 \ge 1-u-2b$ and substituting into R,

$$R(a_n) \;\le\; VS_0^{(n)} + V\mathcal{J}_p + VE_3 \;+\; \sum_i Q_i^+\Big[(u+2b)\big(\lambda_i^{\mathrm{pkt}}+\varepsilon\big) - \varepsilon\Big], \qquad E_3 := e_1+e_2+b\bar\psi_{\max}.$$

The slack condition of Theorem 5.1 gives $(u+2b)(\lambda_i^{\mathrm{pkt}}+\varepsilon) \le \varepsilon/2$ for every i, hence

$$R(a_n) \;\le\; VS_0^{(n)} \;-\; \frac{\varepsilon}{2}\,S_0^{(n)} \;+\; V\mathcal{J}_p \;+\; VE_3. \tag{33}$$

This is the accounting demanded by the queue-dependent bias terms: the factors u and b multiply queue sums only inside $(u+2b)(\lambda_i^{\mathrm{pkt}}+\varepsilon)\sum_i Q_i^+$, which the ε-slack cancels at the epoch level, while V is only ever multiplied by the state-free constants in E_3. No stationary queue moment is ever multiplied by V.

Step 5: Exact cancellation of $VS_0^{(n)}$. *The score used $\hat\tau$; the cost identity uses $\bar m_n$; the two were reconciled once, in Step 4, and nowhere touch the backlog term.* Substituting (33) into (30) and expanding $V\,\mathbb{E}[C_n \mid \mathcal{F}_n]$ on the left with Lemma F.2, the variance terms $\frac{V\Lambda}{2}\mathrm{Var}(\tau_n \mid \mathcal{F}_n) =$

$V\bar{m}_n v(a_n)$ cancel exactly, and the terms $V\bar{m}_n S_0^{(n)}$ cancel exactly, because frame cost and master inequality carry the *same* multiplier $\bar{m}_n$; the estimation bias never multiplies $S_0^{(n)}$. What remains is

$$\mathbb{E}[\Delta_n \mid \mathcal{F}_n] \;+\; V\bar{m}_n\Big(\psi(a_n) - \mathcal{J}_p - E_3\Big) \;\le\; \tilde{B}_0' \;-\; \frac{\varepsilon}{2}\,\bar{m}_n\, S_0^{(n)} \;\le\; \tilde{B}_0'. \tag{34}$$

Step 6: Telescoping and normalization. Define R-RDPP's duration-weighted average inefficiency over the first n_0 frames, n_0 deterministic; since $\psi(a_n)$ and $v(a_n)$ are $\mathcal{F}_n$-measurable, the tower property gives $\mathbb{E}[\tau_n\psi(a_n)] = \mathbb{E}[\bar{m}_n\psi(a_n)]$ and likewise for v, so

$$\bar{\psi}^{\text{R-RDPP}} := \limsup_{n_0\to\infty} \frac{\sum_{n<n_0}\mathbb{E}\big[\tau_n\,(\psi(a_n)+v(a_n))\big]}{\sum_{n<n_0}\mathbb{E}[\tau_n]};$$

the deterministic frame count avoids any optional-stopping argument, and since $1 \le \tau_n \le \bar{\tau}$ the frame-count and slot-horizon averages coincide in the limit. Summing (34) over $n < n_0$, taking expectations, dropping $\mathbb{E}[\mathcal{L}(Q^+(T_{n_0}))] \ge 0$, dividing by $V\sum_{n<n_0}\mathbb{E}[\tau_n] \ge Vn_0$, the initial condition $\mathcal{L}(Q^+(0))/\big(V\sum_{n<n_0}\mathbb{E}[\tau_n]\big)$ vanishes as $n_0 \to \infty$ and

$$\limsup_{n_0\to\infty} \frac{\sum_{n<n_0}\mathbb{E}[\tau_n\psi(a_n)]}{\sum_{n<n_0}\mathbb{E}[\tau_n]} \;\le\; \mathcal{J}_p + E_3 + \frac{\tilde{B}_0'}{V}.$$

For the surcharge, $\mathbb{E}[\tau_n v(a_n)] = \frac{\Lambda}{2}\mathbb{E}[\mathrm{Var}(\tau_n \mid \mathcal{F}_n)] \le \Lambda\Delta_g^2/8$ at nonempty epochs and 0 at empty ones, while every nonempty frame lasts at least $\tau_{\min}$ slots, so its duration-weighted average is at most $\Lambda\Delta_g^2/(8\tau_{\min})$. Taking the infimum over ε-slack grid comparators,

$$\bar{\psi}^{\text{R-RDPP}} \;\le\; J_\delta^\star \;+\; \frac{\tilde{B}_0'}{V} \;+\; \frac{\Lambda\,\Delta_g^2}{8\,\tau_{\min}} \;+\; E_3, \qquad E_3 = \Big(\frac{\eta\Delta_g+\beta}{\tau_{\min}}\Big)\Big(\bar{\psi}_{\max} + \frac{\Lambda(\bar{\tau}-1)}{2}\Big) + 2\beta c_\psi, \tag{35}$$

where the closed form of E_3 follows by collecting $e_1 + e_2 + b\bar{\psi}_{\max}$. Since $\beta = \Delta_g + \kappa\delta$, the four error terms are respectively $O(1/V)$, $O(\Delta_g^2)$, $O((1+\eta)\Delta_g)$, and $O(\delta)$, as claimed.

Step 7: Backlog bound. Retaining the slack term in (34) and using $|\psi(a_n)| \le \bar{\psi}_{\max}$, $\mathcal{J}_p \le \bar{\psi}_{\max}$, and $\bar{m}_n \le \bar{\tau}$,

$$\mathbb{E}[\Delta_n \mid \mathcal{F}_n] \;\le\; \tilde{B}_0' + V\bar{\tau}\big(2\bar{\psi}_{\max} + E_3\big) \;-\; \frac{\varepsilon\,\tau_{\min}}{2}\sum_i Q_i^+(T_n),$$

a Foster–Lyapunov drift condition with a $\Theta(V)$ constant. Telescoping it and passing from epochs to slots exactly as in Steps 4–5 of the proof of Lemma 4.1 (bounded frame lengths, per-frame arrival envelope) yields, for every fixed V,

$$\limsup_{T\to\infty}\frac{1}{T}\sum_{t=0}^{T-1}\mathbb{E}\Big[\sum_i Q_i(t)\Big] \;=\; O(V) \;<\; \infty,$$

which completes the proof. □

Remark 3 (Why saturated scoring). *Lemma E.1(b) is one-sided: only for a saturated action is the reference-profile estimate $\hat{\tau}(L)$ within β of the realized conditional mean. If Algorithm 2 scores a threshold L whose top-k batch has realized grid maximum $\tilde{L} < L$, the declared estimate can exceed $\bar{m}$ by up to $\kappa(L-\tilde{L})+\Delta_g$, which is not $O(\beta)$, and the score's service credit $V(\hat{\tau}-1)d/\hat{\tau}$ then overstates the true credit by a first-order amount: the estimated score is increasing in the declared $\hat{\tau}$ whenever $Vd - W_Q > V\Lambda\hat{\tau}^2/2$, so an overshoot up to $\hat{\tau} \approx \sqrt{2d/\Lambda}$ can be score-profitable while physically wasteful. Saturated scoring removes the mismatch. Alternatively, for a variant of Algorithm 2 that omits the saturation padding step, the bound of Theorem 5.2 holds with the same constants whenever $\Lambda\,\tau_{\min}^2 \ge 2kt_{\max}^{in}$: then $\Lambda/2 \ge d/(\hat{\tau}\bar{m})$ for every action, the overshoot term in the proof of Lemma F.3(ii) is nonpositive, and the conversion holds for unsaturated actions as well.*

Remark 4 (The benchmark must be causal). *No additive bound of the form* (35) *holds against a* clairvoyant *comparator that observes output lengths before committing. Consider two stations holding prompts with identical inputs whose decode durations independently take two values Δ_g apart, each with probability $1/2$, and a batch budget of one station per frame. The clairvoyant comparator always transmits the station whose realized decode is short, so its mean frame duration is shorter by $\Theta(\Delta_g)$, and by the $\Lambda(\bar{m}-1)/2$ term its inefficiency rate is smaller by $\Theta(\Delta_g)$ per frame. Any causal policy, R-RDPP included, sees exchangeable $\mathcal{F}_n$-statistics for the two stations and cannot correlate its choice with the realized decode, so it forfeits this $\Theta(\Delta_g)$ rate advantage on every frame. An additive gap of order Δ_g against the clairvoyant benchmark is therefore unavoidable for every causal policy, and the offered-service comparator class of Definition F.1, which is causal by construction, is the appropriate benchmark; this replaces the output-observing oracle framing used in an earlier version of Theorem 5.2.*

Discussion of Theorem 5.2. Theorem 5.2 extends the frame-efficiency guarantee of Theorem 4.2 to output uncertainty and continuous input lengths, and separates four sources of sub-optimality. The $O(1/V)$ term is the algorithmic drift-plus-penalty gap and can be made arbitrarily small by increasing V. The $O(\Delta_g^2)$ term is the variance surcharge v: a causal scheduler commits to a batch before output lengths are revealed, so every frame carries the conditional decode variance, which Popoviciu's inequality bounds by $\Delta_g^2/4$ (Lemma E.1(c)).

The $O((1+\eta)\Delta_g)$ term collects the estimation bias of the provisioned reference profile (Lemma E.1) and the score distortion of the robustness weighting (Lemma E.2); it vanishes when decode durations are deterministic. The $O(\delta)$ term is the quantization bias $\kappa\delta$ and vanishes as $\delta \to 0$; at $\Delta_g = 0$ and $\delta = 0$ the bound recovers Theorem 4.2.

The benchmark is deliberately causal: Remark 4 in Appendix F shows that against a clairvoyant comparator that observes output lengths before committing, no additive bound of this form exists, because clairvoyance confers a $\Theta(\Delta_g)$ per-frame efficiency advantage; the offered-service class therefore provides an appropriate causal benchmark for the present guarantee, replacing the oracle framing used in an earlier version of this theorem. As in Theorem 4.2, no claim is made that the total cost is within $O(1/V)$ of the causal optimum; the efficiency benchmark is the component of the cost that the score provably controls.